

# Accurate, Secure, and Efficient Semi-Constrained Navigation with Multiple Spatial Restrictions

**Abstract**—Graph-Based Service (GBS) facilitates navigation in multiple scenarios. Although convenient, it requires users to upload their locations to an untrusted navigation server. Such interaction violates location privacy of users who prioritize sensitive locations. Unfortunately, existing work neglects partial visit orders and geographic features, which motivates us to consider *partially ordered stops* and *circumtous path* between two stops. In this work, we advance the state-of-the-art on Secure Semi-Constrained Navigation by allowing Multiple Spatial Restrictions (SCN-MSR), while improving navigation accuracy without sacrificing security. Based on the Divide-and-Conquer strategy, we design four novel navigation algorithms NLow, NMid, NHigh, and secure skimming. They enable an untrustworthy navigation server to search over an encrypted dataset to find the shortest path via  $k$  stops, subject to following orders of several stops and circumventing road obstacles. Next, we provide formal security analysis under the semi-honest adversary model. Experimental results demonstrate that Gaia achieves high geometric alignment with theoretical optimal paths and reduces navigation time by approximately 60% compared to the state-of-the-art Hermes, requiring only about 1.6 s to search over 2,200 road intersections with 5 stops and 3 partial orders.

**Index Terms**—Navigation, Unordered Stops, Spatial Restrictions, Security, Accuracy, Efficiency.

## I. INTRODUCTION

### A. Background

Graph-Based Service (GBS) has flourished in recent years, facilitating graph-structured services, such as social search [1], communication network search [2] online web advertising [3], and navigation services [4]. Among different types of GBS, Shortest Path Computation (SPC) is crucial to both daily life and exceptional circumstances. For example, Google Maps and Waze are two of the most widely used navigation tools today, which enable users to get directions from a starting point  $S$  to a destination  $D$ . Meanwhile, military supply transport, disaster relief shipping, and critical medicine delivery also demand for SPC to improve efficiency.

In SPC, navigation users have to upload their real-time locations or entered locations to a Navigation Server (NS), which is computationally powerful. But NS is proven and assumed to be not trusted [5]–[9]. Such a common assumption further highlights the importance of location privacy since the leakage of sensitive location information can result in advertising harassment, social connection exposure, and persistent surveillance [10].

### B. Existing Work

To address the location privacy leakage, Meng et al. [7] proposed a graph encryption scheme GRECS for approximate shortest distance queries. Following this work, there are

several well-studied graph encryption-based schemes, such as graph encryption for shortest distance queries (SecGDB) [11], graph encryption for shortest path queries (SPQ) [12], privacy-preserving shortest distance queries (GENOA) [16], large graphs for secure queries (GraphShield) [13], and privacy-enhancing and accurate shortest path retrieval (Crypt-Graph) [14], PathGES [15]. Their key idea is to query a shortest distance or path in an encrypted graph while protecting location privacy.

In this work, we focus on unraveling new problems on securely navigating via  $k$  unordered stops based on the seminal paper in this line of research, i.e., the state-of-the-art (sota) Semi-Constrained Navigation (SCN) scheme Hermes [17]. Its main idea is to preprocess and encrypt the road graph and send a query token of  $k$  stops to the NS, which computes the shortest path over location ciphertexts. Especially, Hermes designs eight secure navigation algorithms by utilizing cryptographic techniques. Its technical crux is to leverage a **Divide-and-Conquer (DaC) navigation approach** to divide the service area into three main areas  $MA^L = \{A_1, A_2\}$ ,  $MA^M = \{A_3, A_4\}$ , and  $MA^H = \{A_5, A_6\}$  using the vector SD and its perpendiculars. Each area implies a distinct visit order. The algorithm applies DaC: entering a sub-area treats it as a new, smaller navigation problem, recursively invoke seven navigation algorithms until the destination is reached.

### C. Problems and Motivations

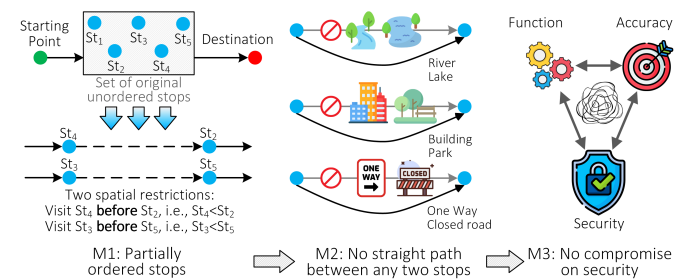


Fig. 1: Three Motivations behind This Work.

Now we discuss the Problems of existing work and present our Motivations while visualizing them in Fig. 1.

**P1: Negligence of partial visit orders.** The SCN fails to account for one actual demand that navigation users have Multiple Spatial Restrictions (MSRs) on the visiting order of two stops among  $k$  unordered stops. **M1 (Function-wise): Partially ordered stops.** For example, Alice has a set of stops  $\{St_1, St_2, St_3, St_4, St_5\}$  and she expects to visit  $St_4$  before

$St_2$  and visit  $St_3$  before  $St_5$ . We use  $St_4 < St_2$ ,  $St_3 < St_5$  to denote the two spatial restrictions. We use Preceding Stop (PSt) and Succeeding Stop (SSt) to denote  $St_4$  ( $St_3$ ) and  $St_2$  ( $St_5$ ), respectively.

**P2: Negligence of geographic features.** First, the core design philosophy of Hermes' navigation strategy is assuming there is a straight path between two stops, thus not considering some real-world navigation obstacles between two locations on a map, e.g., river, lake, building, park, road closure, and one-way road. Second, the Vertically Nearest Hop (VNH) strategy from the DaC solution [17] is not entirely correct because it computes the first Bridge Point (BP) in  $MA^M$  from the perspective of the starting point, rather than the last visited stop in  $MA^L$ . **M2 (Accuracy-wise): Circuitous path between two stops.** First, we embrace the fact that there is no straight path between two locations since there are different types of obstacles. We use  $St_1 \leftrightarrow St_2$  to denote a road closure between  $St_1$  and  $St_2$ . We use current stop (CSt) and next stop (NSt) to denote  $St_1$  and  $St_2$ , respectively. Second, we retain the geographic blocks between any two intersections (stops) and accept the impact of such features on the navigation.

**M3 (Security-wise): No compromise on security.** When realizing a new function and improving navigation accuracy, our new endeavour cannot sacrifice security. First, neither the navigation query nor the route computation can leak the partial order to the NSP. Second, both the navigation query and the route computation must not reveal to the NSP any partial information about the block between any two stops.

For the simple version of M1, i.e., only one spatial restriction  $St_4 < St_2$  is considered, we have two potential solutions: First, the user can compute the combination of different routes based on the remaining unordered stops, submit corresponding navigation queries to the cloud, concatenate the navigation sub-paths, and compare the path distance of different routes. As in the example above, Alice has a potential route  $S - St_4 - St_2 - D$  and she can insert  $St_1$  and  $St_3$  into the three slots, leading to nine queries. In other words, one spatial restriction on two stops from  $k$  stops will lead to  $(2 + 1)^{k-1}$  queries, which is quite cumbersome and unnecessary. Another solution is to set the subsequently visited stop as a temporary destination and then march toward the original destination. This will get us two queries  $(S, St_2; St_4)$  and  $(St_2, D; \emptyset)$  in the example above. However, the same issue persists since we are not certain how to fit  $St_1$  and  $St_3$  in the navigation, which traps us in the traversal predicament.

For M2, the most intuitive solution is to start from CN, bypass the block, and land on the NSt. This can be done by navigating to one of the (at least) two directions, i.e., left front and right front. Meanwhile, the navigation does not end at NSt when there are still more unvisited stops. Blindly bypassing the block without considering remaining nodes will probably reduce navigation accuracy.

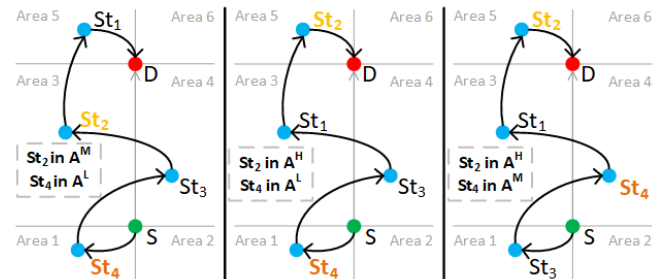
#### D. Our Approach and Technical Challenges

Existing navigation schemes exhibit two functional gaps. First, their pseudo-random permutation/function obscures rel-

ative vectors and distances, failing to support obstacle avoidance. Second, enforcing partial orders incurs an intractable  $O(k!)$  permutation complexity [11]–[15]. To bridge these gaps, we propose *Secure Semi-Constrained Navigation with Multiple Spatial Restrictions*, named *Gaia*, to integrate geometric heuristics with homomorphic encryption to handle both obstacle avoidance and partial order constraints, to fulfill all the motivations.

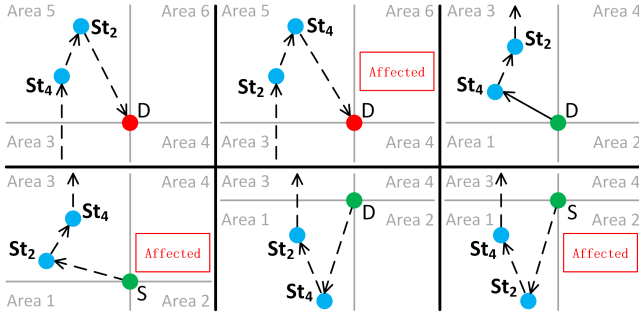
(1) We preprocess and encrypt an original road graph with CKKS (Cheon-Kim-Kim-Song, CKKS) [18]. (2) We observe the locations of PSt and SSt. For instance, we determine which area(s)  $St_2$  and  $St_4$  sit in, i.e.,  $MA_{St_2}$  and  $MA_{St_4}$ , based on the distribution results of grouping strategy from DaC. If  $MA_{St_4} < MA_{St_2}$ , we invoke DaC. (3) During the preprocessing, we set a block indicator between any two nodes in the original dataset and plant a Deviation Stop (DSt) on the shortest path of any two straight-path-blocked stops to bypass the block between them.

*TC1: How to achieve accurate, secure, and efficient through  $k - 2$  unordered stops and 2 ordered stops (PSt, SSt) when  $MA_{PSt} = MA_{SSt}$ ?* The original design of DaC did not consider partially ordered nodes, which not only created challenges for addressing the issue but also presented opportunities for algorithmic improvement. As portrayed in Fig. 2, when  $St_4$ 's area is higher than  $St_2$ 's, i.e.,  $MA_{St_4} < MA_{St_2}$ , the navigation is unaffected. This holds true for case 1, case 3, and case 5 in Fig. 3 where  $MA_{St_4} = MA_{St_2}$  as well (Note that straight paths represent logical point-to-point Euclidean vectors, while the actual physical traversal computes the exact sum of edge weights along the map, which is typically longer than the direct Euclidean distance). But in the other three cases in Fig. 3. These situations become more complex for four reasons. First, hopping to PSt before SSt in such situations conflicts with the existing navigation strategy. Second, there will be other unvisited stops being associated with accurately navigating in current area. Last, given that navigation in the plaintext domain already presents significant challenges, navigation in the ciphertext domain while correctly invoking navigation strategies of DaC, making the first TC more challenging than SCN.



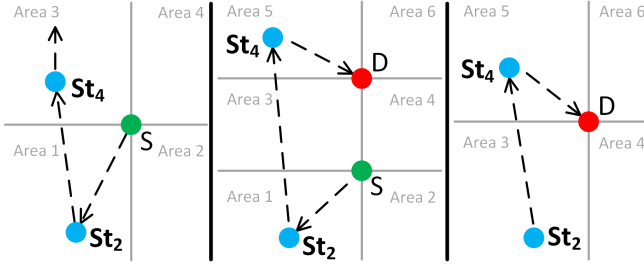
**Fig. 2:** Three Navigation Cases Unaffected by the Partial Visit Order ( $St_4 < St_2$ ) when  $MA_{St_4} \ll MA_{St_2}$ .

*TC2: How to achieve accurate, secure, and efficient SCN-MSR through  $k - 2$  unordered stops and 2 ordered stops (PSt, SSt) when  $MA_{PSt} \gg MA_{SSt}$ ?* As illustrated in Fig. 4,  $MA_{PSt}$



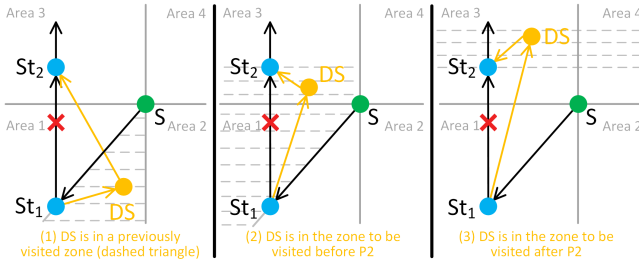
**Fig. 3:** Six Navigation Cases when  $MA_{St_4} = MA_{St_2}$ . Three of Which are Affected by the Partial Visit Order ( $St_4 < St_2$ ).

is higher than  $MA_{Sst}$ . This situation resembles the one in TC1 regarding hopping to PSt prior to SSt, conflicting with area visit priorities. The unique location of  $S$  implies that navigation will be trickier when other stops are situated in the two areas.



**Fig. 4:** Three Navigation Cases Affected by the Partial Visit Order ( $St_4 < St_2$ ) when  $MA_{St_4} >> MA_{St_2}$ .

**TC3:** How to achieve accurate, secure, and efficient SCN-MSR through the planted DSs as an optional and partially ordered stop in different zones? As displayed in Fig. 5, due to the block between  $St_1$  and  $St_2$ , we recruit a DSt as a detour indicator. But the DSt can situate in three areas, which call for different navigation strategies. For example, when standing in the unvisited zone above  $St_2$ , the DSt provides an optional way to hop forward. Nevertheless, the mutual correlations between DSt and the unvisited stops within this zone complicate the navigation. Hence, the DSs make TC3 more challenging than those encountered in prior work.



**Fig. 5:** Three Navigation Cases with  $St_1 \leftrightarrow St_2$  and A (yellow) Deviation Node between  $St_1$  and  $St_2$ .

**TC4:** How to navigate from  $S$  into  $MA^L$  when we do not

have a reference stop in  $MA^M$ ? When the precomputed  $B_1$  is abandoned for the purpose of improving accuracy, we lose a secondary navigation reference location besides  $S$ , disabling SS and SC. Such a predicament prevents us from initiating navigation to  $MA^L$ .

### E. Our Contributions

**For TC1, we formulate a strategy 1area-Wait (1W) when  $A_{Pst} = A_{Sst}$ .** It checks whether the next hop is SSt before hopping to PSt. If not, the navigation continues; otherwise, it puts SSt in the set of visited stops in  $A_{Pst}$ , hops to the next hop until PSt is visited, then puts SSt back to the set of unvisited stops in  $A_{Pst}$ . Note that 1W strategy applies to three cases where both PSt and SSt are in  $MA^H$ ,  $MA^M$ , and  $MA^L$ , respectively. **For TC2, we develop a strategy 2area-Delay (2D) when  $A_{Pst} \gg A_{Sst}$ .** It puts SSt on hold, continues to hop to PSt, then circles back to SSt. **For TC3, we devise a strategy ViaDSt by observing the DSt' location when there is a DSt between the current stop (CSt) and the next stop (NSt).** If it is in a zone that is visited (to be visited before the next stop), there are certainly no other stops in the zone, we hop backward (forward) to DSt, then to NSt. Last, if it is in a zone to be visited after the next stop, we hop forward to DSt, then to NSt if there are no other stops. Otherwise, we draw an auxiliary line from CSt to NSt to observe their locations and decide which one to hop to. If the navigation advances clockwise, we hop to the leftmost stop located on the left (right) of  $L_{CSt}^{NSt}$ . If the navigation advances anticlockwise, we proceed similarly by changing the leftmost to the rightmost.

**For TC4, we plan a strategy C-AC (Clockwise-AntiClockwise), i.e., navigating by observing the presence of stops in six subareas.** For example, If  $A_3$  has stops while  $A_4$  does not, we navigate into  $A_2$  first before  $A_1$ . Next, we design a new navigation algorithm Secure sKim (SK) to orient  $St_j$  against  $BSt_i$  at most  $n^{semi}$  times in a semispace of  $A^L$ .

Our contributions are summarized as follows.

- **New navigation function for Secure SCN.** We have realized a new navigation function, which allows users to impose multiple spatial restrictions on the  $k$  stops.
- **Improved navigation accuracy for Secure SCN.** We have designed new navigation strategies to ensure **geometric alignment** with the physical shortest path, significantly improving the navigation accuracy over cipher-texts.
- **Formal analysis and evaluation.** We have provided formal security analysis, complexity analysis, and implementations with extensive experiments.

**Paper Organization.** We review related work in Section II. We introduce system model, threat model, and design objectives in Section III. We revisit preliminaries in Section IV. In Section V, we present Gaia followed by formally security analysis in Section VI and performance analysis in Section VII. Finally, we conclude with Section VIII and discuss future work in Section IX.

TABLE I: Comparison with Existing Schemes

| Schemes          | Tow-Cloud Model | Cryptographic Techniques | Location privacy |             | Unordered Stops | Spatial Restrictions | Accuracy    | Search Efficiency |
|------------------|-----------------|--------------------------|------------------|-------------|-----------------|----------------------|-------------|-------------------|
|                  |                 |                          | Source           | Destination |                 |                      |             |                   |
| GRECS [7]        | ×               | HE, PRF, PRP             | ●                | ●           | ×               | ×                    | Approximate | ■■■               |
| SecGDB [11]      | ✓               | HE, PRF, PRP, GC, OT     | ●                | ●           | ×               | ×                    | High        | ■■■               |
| GENOA [16]       | ×               | HE, PRF                  | ●                | ○           | ×               | ×                    | Approximate | ■■■               |
| GKT [12]         | ×               | STE, PRF                 | ●                | ●           | ×               | ×                    | High        | ■■■               |
| GraphShield [13] | ✓               | HE, PRF, PRP, GC, OT     | ●                | ●           | ×               | ×                    | High        | ■■■               |
| CryptGraph [14]  | ✓               | HE, SeS                  | ○                | ○           | ×               | ×                    | High        | ■■■               |
| PathGES [15]     | ✓               | EMM                      | ●                | ●           | ×               | ×                    | High        | ■■■               |
| Hermes [17]      | ✓               | HE, PRF, PRP, GC, OT     | ●                | ●           | ×               | ×                    | Medium      | ■■■               |
| Gaia             | ✓               | HE, PRF, PRP, GC, OT     | ●                | ●           | ✓               | ✓                    | High        | ■■■               |

## II. RELATED WORK

To facilitate secure search over structured data, Chase and Kamara [20] pioneered the notion of Structured Encryption (STE). Meng et al. [7] designed GRECS including three oracle encryption schemes for approximate shortest distance queries by employing Homomorphic Encryption (HE), Pseudo-Random Function (PRF), and Pseudo-random Permutation (PRP). graph encryption for shortest distance queries (SecGDB) [11] Liu et al. [16] proposed GENOA, a 2-hop cover labeling [21] based graph encryption scheme to enable shortest distance queries on an encrypted and remotely stored graph. Ghosh et al. [12] presented the first graph encryption scheme GKT for Single-Pair Shortest Path (SPSP) queries based on a dictionary encryption scheme. Du et al. [13] answered shortest distance queries, other classic graph-based queries (e.g., maximum flow) and more complicated analytics (e.g., PageRank) by designing GraphShield, a structured encryption scheme for graphs by using HE, PRF, PRP, Garbled Circuits (GC), and Oblivious Transfer (OT). CryptGraph [14] designed CryptGraph to achieve efficient and accurate retrieval over encrypted graphs based on HE and Secret Sharing (SeS). It leveraged two non-colluding servers to collaboratively retrieve the shortest path and shortest distance. Although graph nodes,  $S$ , and  $D$  are permuted, still they are processed in plaintext. PathGES [15] is an improvement over GKT [12]. Based on response-hiding Encrypted MultiMap (EMM), it designs a new data structure that mitigates previous attacks while achieving non-interactive computation and fast query time. Hermes [17] introduced the notion of Semi-Constrained Navigation (SCN) and solved the secure SCN by putting forth the Divide-and-Conquer (DaC) navigation approach together with eight meticulously tailored navigation algorithms. We will provide a detailed introduction to DaC in Section IV.

These schemes have slightly different security goals or underlying techniques. Our work aims to achieve accurate, secure, and efficient navigation strategies and algorithms for semi-constrained navigation with multiple spatial restrictions. A comparative summary is provided in Table I.

## III. PROBLEM STATEMENT

### A. System Model

The system model of Gaia is depicted in Fig. 6, consisting of user  $\mathcal{U}$ , navigation server  $\mathcal{S}$ , and a navigation proxy  $\mathcal{P}$ . We

list the key notations in Table II.

In the Initialization phase, the user preprocess a road dataset  $\mathcal{RD}$  to obtain a road graph  $\mathcal{RG}$  and generates cryptographic parameters. In the Encryption phase, the user encrypts  $\mathcal{RG}$  to obtain path-distance oracles of nodes and an encrypted road graph  $\mathcal{EG}$ . The user uploads  $\mathcal{EG}$  to the NS and shares a partial secret key  $sk_1$  from the secret key set  $\mathcal{SK} = \{sk_1, sk_2, sk_3, sk_4\}$  with the navigation proxy. In the Query phase, the user has a navigation query  $nq$  with  $k$  unordered stops  $\{St_1, St_2, \dots, St_k\}$ , generates a query token  $t_{nq}$ , and submits  $t_{nq}$  to the server. In the Search phase, the server searches  $\mathcal{EG}$  by using  $t_{nq}$ , computes an encrypted shortest path  $Esp_{nq}$  with the assistance of the proxy, and returns  $Esp_{nq}$  along with the encrypted shortest distance  $Edt_{nq}$  to the user. In the Decryption phase, the user decrypts the result to recover the shortest path  $sp$  and shortest distance  $dt$  in plaintext.

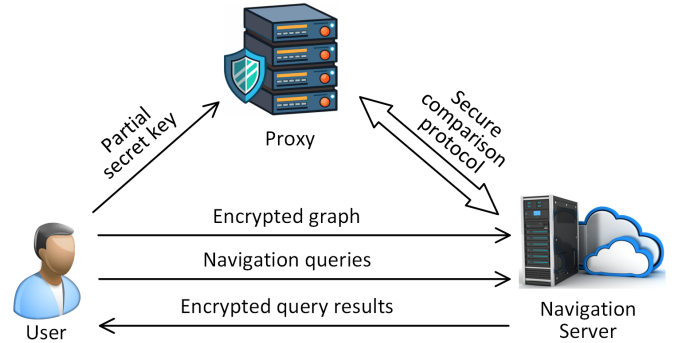


Fig. 6: System Model of Gaia.

We formally define the SCN-MPR and the Secure SCN-MPR as follows.

**Definition 1 (SCN-MPR).** Given a starting point  $S$ , a destination  $D$ , a set of  $k$  stops  $\mathcal{ST} = \{St_1, \dots, St_k\}$ , a set of  $t$  partial orders  $\mathcal{PO} = \{O_1, \dots, O_t\}$ , and a navigation function  $\mathcal{NF}$ , SCN-MPR is to find a sequence  $S$  of  $k$  different stops in  $\mathcal{ST}$  (satisfying  $\mathcal{PO}$ ) constituting a shortest path from



TABLE II: Key Notations of TiMTAPS

| Notation                      | Definition                                      |
|-------------------------------|-------------------------------------------------|
| SCN                           | Semi-Constrained Navigation                     |
| SCN-MSR                       | SCN with Multiple Spatial Restrictions          |
| DaC, $MA, A$                  | Divide-and-Conquer, main area, area             |
| S, D, St, BP                  | Starting point, Destination, Stop, Bridge Point |
| PSt, SSt                      | Preceding Stop, Succeeding Stop                 |
| CSt, NSt                      | Current Stop, Next Stop                         |
| HE                            | Homomorphic Encryption                          |
| PRF, PRP                      | Pseudo-Random Function/Permutation              |
| GC, OT                        | Garbled Circuits, Oblivious Transfer            |
| STE, SYE                      | Structured Encryption, Symmetric Encryption     |
| $U, S, P, \mathcal{RD}$       | User, navigation server, proxy, road dataset    |
| $\mathcal{RG}, \mathcal{ERG}$ | Road dataset, road graph, encrypted road graph  |
| $nq, tk, PO$                  | Navigation query, query token, partial order    |
| $\mathcal{NQ}, \mathcal{ST}$  | Set of navigation queries, set of stops         |
| $n, k$                        | Number of nodes, number of stops                |
| $sp, dt, \mathcal{L}$         | Shortest path/distance, leakage function        |
| $\Sigma, \Omega$              | DaC scheme, CKKS encryption scheme              |
| $F, P, H, h$                  | PRF, PRP, random oracle, hash function          |
| $SK, PK$                      | Secret key, public key                          |
| $\mathcal{ERD}, \mathcal{AR}$ | Road graph dictionary, array                    |
| $Esp, Edt$                    | Encrypted shortest path/distance                |
| $\mathcal{EQR}$               | Encrypted query result                          |
| Nav, NLow                     | Overall algorithm, sub-algorithm for $A^L$      |
| NMid, NHigh                   | Sub-algorithm for $A^M$ and $A^H$               |
| $C, AC$                       | Clockwise, anticlockwise                        |
| $SK, PP$                      | Secure sKimming, perpendicular vector           |

$S$  to  $D$  passing  $\mathcal{ST}$ :

$$\arg \min_{St_{i_1}, \dots, St_{i_k}} \mathcal{NF}(S, D, \mathcal{ST}, \mathcal{PO}) = \{(St_{i_1}, \dots, St_{i_k}) \mid \forall S(St_{j_1}, \dots, St_{j_k}) : (\text{Dist}(S, D, St_{i_1}, \dots, St_{i_k}) \leq \text{Dist}(S, D, St_{j_1}, \dots, St_{j_k})) \wedge (St_{i_1} < St_{i_2}, i \in [t])\}.$$

**Definition 2 (Secure SCN-MPR).** A secure SCN-MPR scheme consists of the following five Probabilistic Polynomial-Time (PPT) algorithms:

- $(SK, PK, \mathcal{RG}, \text{SCGC}) \leftarrow \text{Ini}(1^\lambda, \mathcal{RD})$ : is a probabilistic generation algorithm executed by the user. It takes security parameter  $\lambda$  and a road dataset  $\mathcal{RD}$  as input, and outputs a secret key set  $SK$ , a public key set  $PK$ , a road graph  $\mathcal{RG}$ , and a secure comparison garbled circuit  $\text{SCGC}$ .
- $\mathcal{ERG} \leftarrow \text{Enc}(SK, pk, \mathcal{RG})$ : is a probabilistic algorithm executed by the user. It takes as input a secret key set  $SK$ , a public key  $pk$  (part of  $PK$ ), and a road graph  $\mathcal{RG}$ , and outputs an encrypted road graph  $\mathcal{ERG}$ .
- $tk \leftarrow \text{Query}(SK, nq)$ : is a probabilistic algorithm executed by the user. It takes as input a secret key set  $SK$  and a navigation query  $nq$ , and outputs a query token  $tk = (S, D, \mathcal{ST} = \{St_i\}_{i=1}^k, \mathcal{PO} = \{PO_i\}_{i=1}^t)$ .
- $(\mathcal{EQR}) \leftarrow \text{Search}(SK, PK, tk, \mathcal{ERG})$ : is an interactive protocol executed among the user, the server, and the proxy. It takes a secret key set  $SK$ , a public key set  $PK$ , a query token  $tk$ , and an encrypted road graph  $\mathcal{ERG}$  as input, outputs an encrypted result  $\mathcal{EQR} = (Esp, Edt)$ .
- $\mathcal{QR} \leftarrow \text{Dec}(SK, \mathcal{EQR})$ : is a deterministic algorithm executed by the user. It takes a secret key set  $SK$  and

an encrypted query result  $\mathcal{EQR}$ , and outputs a query result  $\mathcal{QR} = (sp, dt)$ .

## B. Security

Now we provide the security requirements for the secure SCN-MSR scheme. The user is honest and the proxy is trusted [13]. From a high-level view, the security we expect from secure SCN-MSR scheme is that: (1) given  $\mathcal{ERG}$ , no adversary  $\mathcal{A}$  can learn any information about  $\mathcal{RG}$  and (2) given a polynomial number of Query executions for a set of SCN-MSR queries  $\mathcal{NQ} = \{nq_1, \dots, nq_w\}$ , no adversary  $\mathcal{A}$  can learn any partial information about  $nq_i$  or  $\mathcal{RG}$ .

Following [7], [12], [17], [22], [23], we parameterize the security definition with *leakage functions* for three phases of Gaia, namely Initialization, Query, and Search. Meanwhile, we also adopt the notion of *adaptive semantic security* of their works in Gaia.

**Definition 3 (Adaptive Semantic Security).** Let  $\Pi = (\text{Init}, \text{Enc}, \text{Query}, \text{Search}, \text{Dec})$  be a SCN-MSR scheme. We define two probabilistic experiments with a semi-honest adversary  $\mathcal{A}$ , a challenger  $\mathcal{C}$ , a simulator  $\mathcal{S}$ , and three leakage functions  $\mathcal{L}_{\text{Init}}$ ,  $\mathcal{L}_{\text{Query}}$ , and  $\mathcal{L}_{\text{Search}}$ :

**Real $_{\mathcal{A}}(1^\lambda)$ :**

- $\mathcal{A}$  generates a road graph  $\mathcal{RG}$ .
- $\mathcal{C}$  calculates  $(SK, PK, \mathcal{RG}) \leftarrow \text{Ini}(1^\lambda, \mathcal{RD})$  and sends  $\mathcal{ERG}$  to  $\mathcal{A}$ .
- $\mathcal{A}$  produces a set of adaptively chosen SCN-MSR queries  $\mathcal{NQ} = (nq_1, \dots, nq_w)$ . For a navigation query  $nq_i$ ,  $i \in [w]$ ,  $\mathcal{A}$  receives  $tk_i \leftarrow \text{Query}(SK, nq_i)$  from  $\mathcal{C}$ , then  $\mathcal{A}$  and  $\mathcal{C}$  run  $\text{Search}(SK, PK, tk_i, \mathcal{ERG})$ .
- $\mathcal{A}$  computes a bit  $b$  to be output by the experiment.

**Ideal $_{\mathcal{A}, \mathcal{S}}(1^\lambda)$ :**

- $\mathcal{A}$  outputs a road graph  $\mathcal{RG}$ .
- Given  $\mathcal{L}_{\text{Init}}$ ,  $\mathcal{S}$  generate and sends  $\mathcal{ERG}$  to  $\mathcal{A}$ .
- $\mathcal{A}$  produces a set of adaptively chosen SCN-MSR queries  $\mathcal{NQ} = (nq_1, \dots, nq_w)$ . For each  $nq_i$ ,  $i \in [w]$ ,  $\mathcal{S}$  is given  $\mathcal{L}_{\text{Query}}$  and  $\mathcal{L}_{\text{Search}}$ .  $\mathcal{A}$  and  $\mathcal{S}$  simulate Search where  $\mathcal{S}$  acts as a user (proxy) and  $\mathcal{A}$  acts as the server.
- $\mathcal{A}$  computes a bit  $b$  to be output by the experiment.

We say that  $\Pi$  is adaptively  $(\mathcal{L}_{\text{Init}}, \mathcal{L}_{\text{Query}}, \mathcal{L}_{\text{Search}})$ -semantically secure if for all PPT adversaries  $\mathcal{A}$ , there exists a PPT simulator  $\mathcal{S}$  such that

$$|\Pr[\text{Real}_{\mathcal{A}}(1^\lambda) = 1] - \Pr[\text{Ideal}_{\mathcal{A}, \mathcal{S}}(1^\lambda) = 1]| \leq \text{negl}(\lambda).$$

Definition 3 conveys that, given  $\mathcal{ERG}$  and a view of the navigation protocol,  $\mathcal{A}$  cannot acquire any information about  $\mathcal{RG}$  beyond the leakage.

## C. Leakage

**Initiation leakage.** The Initiation leakage includes the number of nodes (in  $\mathcal{RG}$ )  $n$ , the maximum distance  $Mdt$ , and the number of deviation nodes  $n_{dev}$ . **Query pattern leakage.** The query pattern QP indicates whether the points in a navigation query have been queried before. For two navigation queries  $nq_1$  and  $nq_2$ , we define  $\text{Sim}(nq_1, nq_2) = (S_1 = S_2, D_1 =$

$D_2, St_{11} = St_{21}, \dots, St_{1k} = St_{2k}$ ), i.e., whether the starting point, destination, and  $k$  stops in  $nq_1$  match the ones in  $nq_2$ . Let  $\mathcal{NQ} = (nq_1, \dots, nq_w)$  be a set of navigation queries. The query pattern leakage function  $\mathcal{L}_{QP}(\mathcal{RG}, nq_i)$  outputs an  $m \times m$  matrix with entry  $(i, j)$  being  $\text{Sim}(nq_i, nq_j)$ . **Access pattern leakage.** The access pattern AP is the set of indices pointing to files that match the query. The access pattern leakage function  $\mathcal{L}_{AP}(\mathcal{RG}, nq)$  for a road graph  $\mathcal{RG}$  and a navigation query  $nq$  outputs the set of vertex identifiers that are retrieved for responding to  $nq$ .

#### D. Function, Accuracy and Efficiency

We also pay attention to both accuracy and efficiency. **Accuracy.** Given the stop visiting sequence, the SCN-MSR system should achieve high accuracy, regarding a low distance deviation rate and a high path similarity [17]. Such computation requires ground truth and equations, which we will introduce in Section VII. **Efficiency.** Despite the computation over ciphertexts, the SCN-MSR system should achieve acceptable efficiency regarding computational costs and communication overhead of entities as well as scalability with  $n$  and  $k$ .

### IV. PRELIMINARIES

#### A. Homomorphic Encryption

The homomorphic encryption scheme we use in this work is the Cheon-Kim-Kim-Song (CKKS) scheme. It consists of five main algorithms:

**HGen( $\lambda$ ):** given a security parameter  $\lambda$ , chooses a power-of-two  $M = M(\lambda, q_L)$ , an integer  $h = h(\lambda, q_L)$ , an integer  $P = P(\lambda, q_L)$ , and a real value  $\sigma = \sigma(\lambda, q_L)$ ; samples  $s \leftarrow \text{HWT}(h)$ ,  $a \leftarrow \mathcal{R}_{q_L}$ , and  $e \leftarrow \mathcal{DG}(\sigma^2)$ ; samples  $a' \leftarrow \mathcal{R}_{P \cdot q_L}$  and  $e' \leftarrow \mathcal{DG}(\sigma^2)$ ; outputs a secret key  $sk = (1, s)$ , a public key  $pk = (b, a) \in \mathcal{R}_{q_L}^2$  where  $b = -as + e$ , and an evaluation key  $evk = (b', a') \in \mathcal{R}_{P \cdot q_L}^2$  where  $b' = -a's + e' + Ps^2 \bmod P \cdot q_L$ .

**HE( $pk, m$ ):** given the public key  $pk$  and a message  $m$ , samples  $v \leftarrow \mathcal{ZO}(0.5)$  and  $e_0, e_1 \leftarrow \mathcal{DG}(\sigma^2)$ , outputs a ciphertext  $c = v \cdot pk + (m + e_0, e_1) \bmod q_L$ .

**HD( $sk, c$ ):** given the secret key  $sk$  and a ciphertext  $c = (b, a)$ , outputs a message  $b + a \cdot s \bmod q_L$ .

**HAdd( $c_1, c_2$ ):** given two ciphertexts  $c_1$  and  $c_2$ , outputs a new ciphertext  $c_{\text{add}} = c_1 + c_2$ .

**HMul( $c_1, c_2, evk$ ):** given two ciphertexts  $c_1 = (b_1, a_1)$  and  $c_2 = (b_2, a_2) \in \mathcal{R}_{q_L}^2$ , sets  $d_0, d_1, d_2 = (b_1 b_2, a_1 b_2 + a_2 b_1, a_1 a_2) \bmod q_L$  outputs a new ciphertext  $c_{\text{mul}} = (d_0, d_1) + \lfloor P^{-1} \cdot evk \rfloor \bmod q_L$ .

#### B. Pseudo-Random Function, Pseudo-Random Permutation

We say that a polynomial-time computable function  $\mathcal{F} : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \{0, 1\}^*$  is a PRF if its output cannot be distinguished from the output of a random function  $f$  by any PPT distinguisher  $\mathcal{D}$ , i.e.,  $|\Pr[\mathcal{D}^{\mathcal{F}_k(\cdot)}(1^n) = 1] - \Pr[\mathcal{D}^{f(\cdot)}(1^n) = 1]| \leq \text{negl}(\lambda)$  [30]. Simply put,  $\mathcal{D}$  can access an oracle  $\mathcal{PO}$  being either  $\mathcal{F}$  or  $f$ .  $\mathcal{D}$  can query  $\mathcal{PO}$  at any point  $i$  to get  $\mathcal{PO}(i)$ . For every  $\mathcal{D}$  that receives a description of  $\mathcal{F}$  outputs 1 with almost the same probability as when it

receives a description of a random function. A PRF is assumed to be a PRP when it is bijective.

#### C. Garbled Circuits and Oblivious Transfer

Yao introduced GC [24], [25] for secure two-party computation. The first party  $C$  called circuit generator creates a circuit  $\tilde{C}$  with wires. For each wire  $w_i$ ,  $C$  chooses two random garbled values  $\tilde{w}_i^0, \tilde{w}_i^1$ , where  $\tilde{w}_i^j$  is the garbled value of  $w_i$ 's value  $j$ . For each gate  $g_i$ ,  $C$  creates a garbled table  $\tilde{T}_i$  that allows to obtain only the garbled value of the corresponding  $g_i$ 's output given a set of garbled values of  $g_i$ 's inputs.  $C$  sends the garbled tables, i.e.,  $\tilde{C}$ , to the second party called circuit evaluator  $E$ .  $E$  obtains the garbled inputs  $\tilde{w}_i$  corresponding to inputs of both parties via oblivious transfer. It evaluates  $\tilde{C}$  on the garbled inputs to obtain the garbled outputs simply by evaluating the garbled circuit gate by gate, using  $\tilde{T}_i$ .

$\text{OT}_l^t$ , parallel 1-out-of-2 OT of  $t$   $l$ -bit strings [26], is a two-party protocol executed between a chooser  $A$  and a sender  $B$ . For  $i = 1, 2, \dots, t$ ,  $B$  inputs a pair of  $l$ -bit strings  $s_i^0, s_i^1$  and  $A$  inputs  $t$  chosen bits  $b_i$ . Upon the completion of the protocol,  $A$  acquires the chosen strings  $s_i^{b_i}$  but not the unchosen strings  $s_i^{1-b_i}$ , and  $B$  learns nothing about  $b_i$ .

#### D. Divide-and-Conquer

The general idea of DaC [17] is to first divide the  $k$  stops into three areas ( $MA^L, MA^M, MA^H$ ) based on the position of  $S$  and the position of  $D$ , determine two bridge points  $B_1$  and  $B_2$  to connect the navigation in three areas, invoke three navigation strategies for the three areas. After each hop, DaC re-groups the stops in current area and starts the recursion. Next, we briefly introduce the navigation strategies and algorithms.

**Secure Grouping (SG).** Given  $S$  and  $D$ , the NS calculates a vector  $\mathbf{SD}$  and the Cosine Similarity (CoSi) of  $\mathbf{SD}$ ,  $\mathbf{SSt}$  and  $\mathbf{DSt}$ ,  $\mathbf{DSt}$ . Given two vectors of attributes  $\mathbf{A}$  and  $\mathbf{B}$  sharing an angle  $\theta$ , their CoSi is defined as:

$$\text{Sim}(\mathbf{A}, \mathbf{B}) = \text{Cos}(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}. \quad (1)$$

To simplify the grouping on  $St_i$ , only the sign of the Numerator of CoSi is required. The NS computes a ciphertext:

$$\begin{aligned} \text{ENC}_{\mathbf{SSt}_i}^{\mathbf{SD}} &= \llbracket (x_{st_i} - x_s)(x_d - x_s) + (y_{st_i} - y_s)(y_d - y_s) \rrbracket_1 \llbracket r \rrbracket_1 \\ &= \llbracket (x_{st_i} - x_s)(x_d - x_s) + (y_{st_i} - y_s)(y_d - y_s) + r \rrbracket_1. \end{aligned} \quad (2)$$

Next, the NS performs comparison on  $\text{NCS}_{\mathbf{SSt}_i}^{\mathbf{SD}}$  ( $\text{NCS}_{\mathbf{DSt}_i}^{\mathbf{SD}}$ ) and  $\llbracket r \rrbracket$  with the proxy based on GC and OT to insert  $St_i$  into one of the three areas.

**Secure Bridging (SB).** After SG, the NS computes a bridge point  $B_1$  as the stop that is vertically nearest to  $S$  in  $MA^M$ . This is done by leveraging the Euclidean distance and CoSi to check all the stops in  $MA^M$  and compute the difference between two projection of two vectors ( $\mathbf{SSt}_i, \mathbf{SSt}_j$ ) on  $\mathbf{SD}$ :

$$\begin{aligned} &\text{Euc}(S, St_i) \text{Sim}(\mathbf{SSt}_i, \mathbf{SD}) - \text{Euc}(S, St_j) \text{Sim}(\mathbf{SSt}_j, \mathbf{SD}) \\ &= \frac{(x_{st_i} - x_{st_j})(x_d - x_s) + (y_{st_i} - y_{st_j})(y_d - y_s)}{\sqrt{(x_d - x_s)^2 + (y_d - y_s)^2}} \end{aligned} \quad (3)$$

Next, the NS sets  $D$  as the second bridge point  $B_2$ .

**Secure Orient-Split-sCan-Hop (OSCH)** for  $MA^L$ . The strategy works for navigation in  $MA^L$ .

- **Secure Orienting (SO)**. The NS orients  $B_1$  and each stops in  $ST^L$  to see which side of  $SD$  they locate in.
- **Secure Splitting (SS)**. After SO, the NS splits the stops in the first semispace of  $A^L$  into two parts  $ST_L^{semi}$ ,  $ST_R^{semi}$  based on the vector  $B_1S$ .
- **Secure Scanning (SC)**. After SP, the NS scans the  $St$  with the maximum or minimum angle  $\angle StB_1S$  when stops are on  $SB_1$ 's right or left side.

**Secure Vertically Nearest Hop (SVNH)** for  $MA^M$ . The strategy works for navigation in  $MA^M$ . It is already introduced in the computation of  $B_1$ .

**Secure Orient-Split-Pull-Hop (OSPH)** for  $MA^H$ . The strategy works for navigation in  $MA^H$ .

- **SO** is the same to one in OSCH.
- **SS** is similar to the one in OSCH except the reference vector is now  $B_2St'$  where  $St'$  is the last visited stop in  $MA^M$ .
- **Secure Pulling (SP)** is the same to the one in OSCH.

We refer the interested reader to [17] for more details.

## V. THE PROPOSED NAVIGATION SCHEME GAIA

### A. Overview

Our scheme Gaia II consists of five phases: Initialization, Encryption, Query, Search, and Decryption. The technical crux is elaborated in Search. Although our navigation rationale follows the DaC, we revise its algorithms to support MSR and optimize them to improve navigation accuracy. II is built upon six building blocks as follows: a DaC scheme  $\Sigma = (SG, SB, SO^L, SP^L, SC^L, SVNH, SO^H, SS^H, SP^H)$ , a CKKS encryption scheme  $\Omega = (HGen, HE, HD, HAdd, HMul)$ , a PRF  $F : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ , a PRP  $P : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ , a random oracle  $H : \{0, 1\}^* \rightarrow \{0, 1\}^*$ , and a collision-resistant hash function  $h$  modeled as a random function [7]. Since we replace the BGN encryption scheme [27] used in  $\Sigma$  [17], some algorithmic changes are necessary, e.g.,  $Esd = Esd * Esd'$  is now  $Esd = Esd + Esd'$ .

### B. Initialization

$\text{Ini}(1^\lambda, \mathcal{RD})$  is run to generate  $SK = (k_1, k_2, k_3, k_4, sk)$ ,  $\mathcal{PK} = pk$ ,  $\mathcal{RG} = (\mathcal{V}, \mathcal{E})$ , and a secure comparison garbled circuit SCGC. Here,  $\mathcal{V}$  is the set of node (intersections),  $\mathcal{E}$  is the set of undirected edges, and  $l_{v_i v_j}$  is the length for each edge  $(v_i, v_j) \in \mathcal{E}$ . The user shares  $sk$  with the proxy through a secure channel. SCGC is invoked by the comparison in  $\Sigma$ .

### C. Enhanced Floyd-Warshall Preprocessing with Spatial Constraints

Our scheme extends the traditional Floyd-Warshall algorithm to support spatial constraints processing. During this phase, we implement comprehensive processing for both blocked edges and partial order constraints.

The blockage handling mechanism marks blocked edges during preprocessing and precomputes DSt through a three-layer strategy to ensure path connectivity. This approach guarantees viable solutions under various blockage scenarios while fully utilizing Floyd precomputation results to avoid unnecessary global searches and achieve intelligent candidate node screening. We present the **algorithm DStSearch** in **Algorithm 1**. The three-layer strategy is detailed as follows:

**Layer 1: Direct DSt Search** → **Layer 2: Neighbor Path Query** → **Layer 3: History Backtracking**

- **Layer 1**: Identifies intermediate nodes that can directly connect blocked edges  $i \rightarrow j$ , screening candidate nodes through vertical projection distance, let  $\mathcal{U}$  denote the set of nodes that have been already used.
- **Layer 2**: If Layer 1 fails, queries paths through neighbors of the blocked edge's starting point, utilizing Floyd precomputation results to directly retrieve shortest paths.
- **Layer 3**: If both previous layers fail, backtracks to previous nodes in the path history  $\mathcal{H}$  for re-planning.

To control computational overhead, the search depth for neighbor exploration in Layer 2 and history backtracking in Layer 3 is strictly regulated.

---

### Algorithm 1 DStSearch

---

**Require:**  $i \rightarrow j, \mathcal{U}, \mathcal{H}$

**Ensure:**  $DSt$

```

1:  $DSt \leftarrow \text{SecureSkimming}(i, j, \mathcal{U})$ 
2: if  $DSt \neq -1 \wedge \text{Connected}(i, DSt, j)$  then
3:   return  $DSt$ 
4: end if
5:  $\mathcal{C} \leftarrow \emptyset$ ; // Candidates list
6: for each  $neighbor \in \text{Adjacency}(i) \wedge neighbor \notin \mathcal{U}$  do
7:   if  $\text{PathExists}(neighbor, j)$  then
8:      $distance \leftarrow \text{Dist}(i, neighbor) + \text{Dist}(neighbor, j)$ 
9:      $path \leftarrow \text{Build}(i, neighbor, \text{GetPath}(neighbor, j))$ 
10:     $\mathcal{C}.\text{add}(\text{Candidate}(neighbor, distance, path))$ 
11:   end if
12: end for
13:  $\mathcal{C}.\text{sortByDistance}()$ ; // Shortest first
14: if  $\mathcal{C} \neq \emptyset$  then
15:   return  $\mathcal{C}[0].\text{firstHop}$ ;
16: end if // Both Layer 1 and Layer 2 failed, proceed to
    backtracking
17: if  $|\mathcal{H}| \geq 2$  then
18:    $prev \leftarrow \mathcal{H}[|\mathcal{H}| - 1]$ ; // Previous node in path history
19:    $newU \leftarrow \mathcal{U} \cup \{i\}$ 
20:   Repeat lines 1-16 with  $(prev, j, newU, \mathcal{H})$ 
21: end if
```

---

### D. Encryption

Given  $\mathcal{RG}$ , the user computes the shortest path  $sp$  and shortest distance  $sd$  for each pair of nodes via the Floyd-Warshall algorithm [28]. Meanwhile, the user records a deviation node  $St_{v_i v_j}^{dev}$  for  $(v_i, v_j)$  if there is no straight path between  $v_i$  and

$v_j$ . All such deviation stops are statically pre-computed offline to ensure fast online query responses without adding real-time computational burden. Next, the user encrypts  $\mathcal{RG}$  by encrypting the information above as follows. For each node  $v_i$  with a pair of coordinates  $(x, y)$ , the user randomly chooses a secret  $k_{v_i}$  and a number  $r_{v_i}$ . For each pair of  $(sp, sd)$  of  $v_i$ , the user creates an array  $\mathcal{AR}_{v_i}$  with the item in the initial location

$$\mathcal{AR}_{v_i}[0] = ((\text{HE}(pk, x) \parallel \text{HE}(pk, y)) \oplus H(k_{v_i} \parallel r_{v_i}), r_{v_i}). \quad (4)$$

The user computes a permutation of  $N$  nodes  $(P(sk_1, v_1), \dots, P(sk_1, v_n))$  to randomize the entry order of  $n$  nodes. For each location  $P(sk_1, v_j)$ , the user chooses a random number  $r_{v_j}$  and stores at  $\mathcal{AR}_{v_i}[P(sk_1, v_j)]$ :

$$(sp_{v_i v_j} \oplus F(sk_3, h(r_{v_j})) \parallel \text{Esd}_{v_i v_j} \parallel \text{E}(St_{v_i v_j}^{\text{dev}})) \oplus H(k_{v_i} \parallel r_{v_j}), r_{v_j}),$$

where  $\text{Esd}_{v_i v_j} = \Omega. \text{HE}(pk, sd_{v_i v_j})$ .

Then, the user creates a road graph dictionary  $\mathcal{RGD}$  to record pairs  $(P(sk_2, v_i), \text{add}_{v_i} \parallel k_{v_i} \oplus F(sk_4, v_i))$  for all  $v_i \in \mathcal{V}$ , where  $\text{add}_{v_i}$  is the location of  $\mathcal{AR}_{v_i}$ . Finally, the user sends  $\mathcal{ERG} = (\mathcal{RGD}, \mathcal{AR})$  to the server. We depict an example of storing road information in  $\mathcal{ERG}$  in Fig. 7.

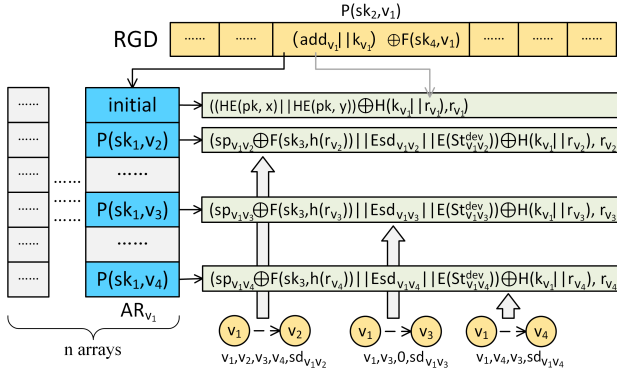


Fig. 7: An Example of Storing Road Information in  $\mathcal{ERG}$ .

### E. Query

To avoid routing conflicts, the user locally strictly validates query inputs by building a directed graph to detect cycles; any cyclic partial order queries are rejected. Once validated, the user has a navigation query:

$$nq = (S, D, St_{i1}, \dots, St_{ik}, PO_{j1}, \dots, PO_{jt}), \quad (5)$$

where  $PO_{jo} = (St_{jo}^1, St_{jo}^2)$ ,  $o \in [t]$  and runs  $\text{Query}(SK, nq)$  to get a query token  $tk$ :

$$tk = (E(S), E(D), E(St_{i1}), \dots, E(St_{tk})), \quad (6)$$

where  $E(S) = (P(sk_2, S), F(sk_4, S))$ ,  $E(D) = (P(sk_2, D), F(sk_4, D))$ ,  $E(St_{io}) = (P(sk_2, St_{io}), F(sk_4, St_{io}))$ ,  $o \in [k]$ .

Later, the user submits  $tk$  to the server and awaits for an encrypted query result.

### F. Search

We give the overview of our navigation strategies in Fig. 8. The overall navigation algorithm Nav is shown in Alg. 2. It consists of four parts. First, it calls Secure Bridging (SB), i.e.,  $\Sigma. \text{SG}(tk) = (E(S), E(D), E(St_{i1}), \dots, E(St_{tk}))$ , to group  $k$  stops into three sets. Next, it invokes three new sub-navigation algorithms, namely NLow, NMid, and NHigh, to determine the next stop in three main areas, respectively. In specific, NLow returns a stop that is used as a starting point for NMid, which does the same for NHigh. Once the next hop is determined, the current algorithm recalls Nav, entering recursion.

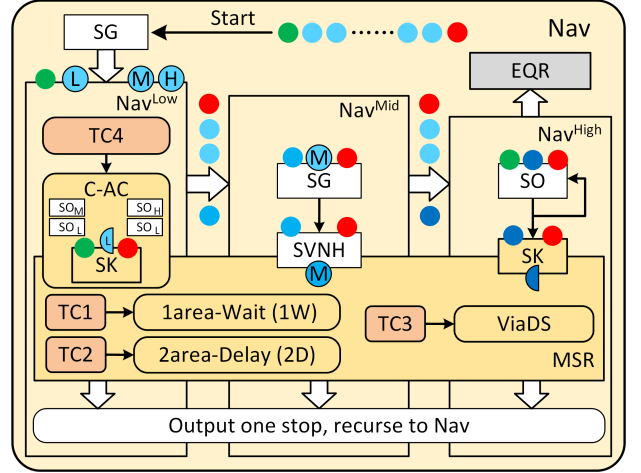


Fig. 8: Overview of Navigation Strategies.

### Algorithm 2 Nav

**Require:**  $\mathcal{ERG}, tk$

**Ensure:**  $\mathcal{EQR}$

- 1: If  $tk = \emptyset$ : Return;
- 2: Parse  $tk = (E(S), E(D), E(St_{i1}), \dots, E(St_{tk}))$
- 3: Initialize  $ST^L, ST^M, ST^H, ST^{1W}, ST^{2D}, \mathcal{EQR}$  as  $\emptyset$ ;
- 4:  $E(St_{last}^L) = \emptyset, E(St_{last}^M) = \emptyset$ ; //two last visited stops
- 5:  $(ST^L, ST^M, ST^H) \leftarrow \Sigma. \text{SG}(tk)$ ; //group the  $k$  stops
- 6: //Below, we omit  $\mathcal{EQR}, \mathcal{EQR}$  for simplicity
- 7:  $E(St_{last}^L) \leftarrow \text{NLow}(E(S), E(D), ST^L, ST^M, ST^H)$ ;
- 8:  $E(St_{last}^M) \leftarrow \text{NMid}(E(St_{last}^L), E(D), ST^M)$ ;
- 9:  $\text{NHigh}(E(St_{last}^M), E(S), E(D), ST^H)$ ;
- 10: Return  $\mathcal{EQR}$ ;

We show the **algorithm NLow** in Algorithm 3. NLow executes our strategy C-AC (lines 3-35). In specific, it starts with observing the node existence  $S_l^M, S_r^M$  of  $A^M$ 's two semispaces (lines 4-10) and grouping stops in  $ST^M$  into two sets  $ST_l^L, ST_r^L$  (lines 11-17) by running  $\Sigma. \text{SO}$ .

Next, NLow proceeds based on the four cases of  $S_l^M$  and  $S_r^M$ , i.e., 10, 01, 11, and 00 (lines 18-30). (1) When  $S_l^M = 1 \wedge S_r^M = 0$ , we first hop from  $S$  to the right semispace of  $A^M$  and to the left semispace of  $A^L$  by invoking a new algorithm **Secure sKimming (SK)**. (2) For  $S_l^M = 0 \wedge S_r^M = 1$ , we hop



to the left semispace of  $A^M$  and to the right semispace of  $A^L$ .  
**(3)** For  $S_l^M = 1 \wedge S_r^M = 1$ , we detect the node existence of  $A^L$ 's two semispaces, and SK with  $C$  and  $AC$  accordingly.  
**(4)** For  $S_l^M = 0 \wedge S_r^M = 0$ , we repeat the process by observing the node existence of  $A^H$ 's two semispaces and  $A^L$ 's two semispaces. Last, NLow updates  $\mathcal{EQR}$  and reignites Nav with current stop  $E(St_{1st}^L)$  and  $D$  (lines 33-35).

---

**Algorithm 3** NLow

---

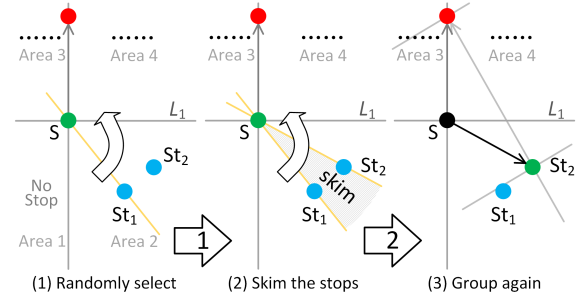
**Require:**  $\mathcal{ERG}$ ,  $E(S)$ ,  $E(D)$ ,  $ST^L$ ,  $ST^M$ ,  $ST^H$ ,  $\mathcal{EQR}$   
**Ensure:**  $E(St_{1st}^L)$ //it is  $E(St_{last}^L)$  when NLow is complete

- 1: **If**  $ST^L == \emptyset$ : return  $E(S)$ ;
- 2: **If**  $|ST^L| == 1$ : execute lines 42-43, return  $E(St) \in ST^L$ ;
- 3: Initialize  $E(St_{1st}^L) = \emptyset$ ;
- 4: Initialize  $S_l^M = 0$ ,  $S_r^M = 0$ ;//two signs for  $A^M$ 's semispaces
- 5: Initialize IT as the first item from  $ST^M$ ;
- 6: **while**  $IT \neq \emptyset$  **do**//assign 0 or 1 to the two signs
- 7:   **if**  $\Sigma.SO(E(S), E(D), IT) == L$ :  $S_l^M = 1$ ;
- 8:   **else**  $S_r^M = 1$ ;
- 9:   Set IT as the next item from  $ST^M$ ;
- 10: **end while**
- 11: Initialize  $ST_l^L = \emptyset$ ,  $ST_r^L = \emptyset$ ;// $A^L$ 's left set and right set
- 12: Initialize IT as the first item from  $ST^L$ ;
- 13: **while**  $IT \neq \emptyset$  **do**
- 14:   **if**  $\Sigma.SO(E(S), E(D), IT) == L$ : add IT into  $ST_l^L$ ;
- 15:   **else** add IT into  $ST_r^L$ ;
- 16:   Set IT as the next item from  $ST^L$ ;
- 17: **end while**
- 18: **if**  $S_l^M = 1 \wedge S_r^M = 0$  **then**//Low right, Low left
- 19:   Retrieve the first item from  $ST_r^L$  as IT;
- 20:    $E(St_{1st}^L) = \text{SK}(E(S), E(D), IT, ST_r^L, C)$ ;//Clockwise
- 21: **else if**  $S_l^M = 0 \wedge S_r^M = 1$  **then**//Low left, Low right
- 22:   Repeat lines 19-20 with  $ST_l^L$ ,  $AC$ ;//AntiClockwise
- 23: **else if**  $S_l^M = 1 \wedge S_r^M = 1$  **then**//look  $A^L$
- 24:   Repeat lines 4-10 with  $S_l^L$ ,  $S_r^L$ ,  $ST^L$ ;
- 25:   **if**  $S_l^L = 1 \wedge S_r^L = 0$  **then**
- 26:     Repeat lines 19-20 with  $ST_l^L$ ,  $C$ ;
- 27:   **else if**  $S_l^L = 0 \wedge S_r^L = 1$  **then**
- 28:     Repeat lines 19-20 with  $ST_r^L$ ,  $AC$ ;//Low right, Clockwise. Excluded 00 in line 1
- 29:   **else**: Repeat lines 19-20 with  $ST_r^L$ ,  $C$ ;
- 30:   **end if**
- 31: **else**: Repeat lines 4-30 with  $S_l^H$ ,  $S_r^H$ ,  $ST_l^H$ ,  $ST_r^H$ ;
- 32: **end if**
- 33:  $Esp \cup = \{(E(St_{1st}^L), sp_{SSt_{1st}^L} \oplus F(sk_3, h(r_{St_{1st}^L})), r_{St_{1st}^L})\}$ ;
- 34:  $E dt + = E sd_{SSt_{1st}^L}$ ;
- 35:  $\text{Nav}(E(St_{1st}^L), E(D), ST_r^L \setminus \{E(St_{1st}^L)\})$ ;

---

Before we dive into SK, we exemplify its navigation rationale with an example in Fig. 9. When there are stops only in Area 2, Area 3, and Area 4, we hop from  $S$  to the right semispace of  $A^L$ . To do so, we randomly select a  $St_1$  from  $ST^L$  and skim another stop  $St_2$  in the remaining stops in this semispace to find the NorthEasternmost stop. We also need to consider two cases where the stops are on line  $L_1$

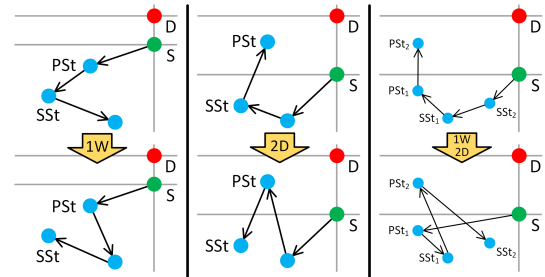
perpendicular to  $SD$  and line  $SS_{St_1}$ . After that, we check the MSRs before hopping to such a stop and reboot Nav.



**Fig. 9:** An Example of Secure sKimming in  $A^L$ .

We show the **algorithm SK** in Algorithm 4. The algorithm is divided into two directions, i.e.,  $C$  and  $AC$ . The **first direction** contains two parts. First, SK computes whether there are stops on line  $L_1$  perpendicular to line  $SD$  (lines 4-9). If  $ST_{L_1}^{On}$  is not empty, we search the nearest stop to  $S$  on  $L_1$  (lines 10-24). Before outputting the next stop  $E(St_1)$ , we first check if it is in the waiting set  $ST^{1W}$ . If  $E(St_1)$  in waiting has a unvisited PSt, SK continues to skim in  $ST$  without  $\{E(St_1)\}$  or MSR; otherwise, SK deletes  $E(St_1)$  from  $ST^{1W}$ . If  $E(St_1)$  is not in  $ST^{1W}$ , we check by 2D. Then, SK checks the multiple spatial restrictions by invoking MSR and returns  $E(St_1)$ . Second, if there are no stops on  $L_1$ , SK continues to skim stops in a clockwise manner and monitor whether  $SS_{St_1}$  has more than one stops (lines 30-36) or not (lines 38-41). The **second direction** resembles the first one, except that we replace  $R$  in line 26 with  $L$  to represent anticlockwise. For instance, when there are stops only in Area 1, Area 3, and Area 4, we hop from  $S$  to the left semispace of  $A^L$ . The other difference is that we look for the NorthWesternmost stop.

Now we introduce the navigation rationale of MSR, which consists of three navigation strategies, i.e., **1area-Wait (1W)**, **2area-Delay (2D)**, and **ViaDSt**. The first two strategies correspond to the first two technical challenges and we give three examples in Fig. 10. For ViaDSt, we show two examples when NSt and DSt are in the same main area, and we need to skim clockwise the leftmost stop from  $CSt$  Fig. 17.



**Fig. 10:** Three Examples of Navigation Strategies (1W, 2D) for Handling Partial Orders.

We show the **algorithm MSR** in Algorithm 5. First, it checks whether the current stop  $St_1$  to be hopped to is a SSt of another stop and they are in the same main area (**1W**, lines

---

**Algorithm 4 SK**


---

**Require:**  $E(S)$ ,  $E(D)$ ,  $IT$ ,  $\mathcal{ST}$ ,  $DR$ ,  $\mathcal{PO}$

**Ensure:**  $E(St)$

```

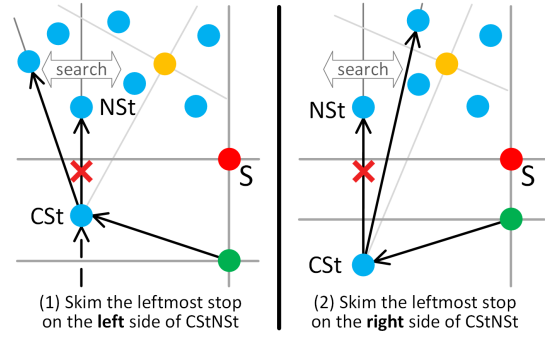
1: If  $|\mathcal{ST}| = 1$ : Return  $E(St) \in \mathcal{ST}$ ;
2: Initialize  $E(St) = \emptyset$ ;
3: if  $DR == C$  then
4:   Initialize  $E(St_i)$  as the first item from  $\mathcal{ST}$ ;
5:   Initialize  $E(St_1)$ ,  $E(St_2)$ , and  $\mathcal{ST}_{L_1}^{On}$  as  $\emptyset$ ;
6:   for each  $E(St_i)$  in  $\mathcal{ST}$  //search stops on  $L_1$ 
7:     Compute  $ENCS_{SS_{t_i}}^{DSt}$  and  $ENCS_{SS_{t_i}}^{DSt}$ ;
8:      $cb \leftarrow SCGC(r, NCS_{SS_{t_i}}^{DSt})$ ;
9:     if  $cb = 1$ : insert  $E(St_i)$  into  $\mathcal{ST}_{L_1}^{On}$ ;
10:  if  $\mathcal{ST}_{L_1}^{On} \neq \emptyset$  then //search the nearest stop to  $S$  on  $L_1$ 
11:    Set  $E(St_1)$  as the first item of  $\mathcal{ST}_{L_1}^{On}$ ;
12:    Set  $E(St_2)$  as the second item of  $\mathcal{ST}_{L_1}^{On}$ ;
13:    While  $E(St_2) \neq \emptyset$ 
14:      Compute  $ENum_{PP_{SD}, S}^{St_1, St_2}$ ,  $ENum_{PP_{SD}, S}^{St_1, St_2}$ ;
15:       $cb \leftarrow SCGC(r, Num_{PP_{SD}, S}^{St_1, St_2})$ ;
16:      //PPSD is a perpendicular vector of SD
17:      if  $cb = 0$ :  $E(St_1) = E(St_2)$ ;
18:      if  $E(St_1) \in \mathcal{ST}^{1W}$ 
19:        if  $OPSt(E(St_1))$  is unvisited; //PSt of  $E(St_1)$ 
20:          Invoke SK without  $\{E(St_1)\}$  or MSR; //1W
21:        else: Delete  $E(St_1)$  from  $\mathcal{ST}^{1W}$ ; //Can visit  $St_1$ 
22:      if  $E(St_1) \in \mathcal{ST}^{2D}$ 
23:        Repeat lines 19-21 with  $\mathcal{ST}^{2D}$ ; //2D
24:      else  $E(St_1) =$ 
25:        MSR( $E(S)$ ,  $E(St_1)$ ,  $\mathcal{ST}$ ,  $\mathcal{PO}$ ,  $\mathcal{ST}^{1W}$ ,  $\mathcal{ST}^{2D}$ )
26:      Return  $E(St_1)$ ;
27:    else //no stop on the line  $L_1$ 
28:      Set  $E(St_2)$  as the second of  $\mathcal{ST}$ ;  $\mathcal{ST}_{SS_{t_1}}^{On} = \emptyset$ 
29:      while  $E(St_2) \neq \emptyset$  do //search the NEmost stop
30:        if  $\Sigma.SO(E(S), E(St_1), E(St_2)) = R$ 
31:           $E(St_1) = E(St_2)$ , Clear  $\mathcal{ST}_{SS_{t_1}}^{On}$ 
32:        else //store the nearest stop to  $S$  on line  $SS_{t_1}$ 
33:          Compute  $ENCS_{SS_{t_2}}^{PP_{SS_{t_1}}, S}$  and  $ENCS_{SS_{t_2}}^{PP_{SS_{t_1}}, S}$ ;
34:           $cb \leftarrow SCGC(r, NCS_{SS_{t_2}}^{PP_{SS_{t_1}}, S})$ ;
35:          if  $cb = 1$ : insert  $E(St_2)$  into  $\mathcal{ST}_{SS_{t_1}}^{On}$ ;
36:          Set  $E(St_2)$  as the next item of  $\mathcal{ST}$ ;
37:        end while
38:      if  $\mathcal{ST}_{SS_{t_1}}^{On} \neq \emptyset$  //search the nearest stop on  $SS_{t_1}$ 
39:        Repeat lines 11-26 with  $PP_{SS_{t_1}}$ ;
40:      else Repeat 25-26; //found one stop not on  $SS_{t_1}$ 
41:    end if
42:  else if  $DR == AC$  then
43:    Repeat lines 4-41 with  $L$  in line 30;
44:  end if

```

---

2-4). If so, MSR inserts  $St_1$  into a waiting set  $\mathcal{ST}^{1W}$  and skims through  $\mathcal{ST} \setminus \{E(St_1)\}$  without MSR. Second, if such a PSt is in a higher main area (2D, lines 5-7), MSR inserts  $St_1$  into a delaying set  $\mathcal{ST}^{2D}$  and skims similarly.

If there is a deviation node between  $E(S)$  (CSt) and  $E(St_1)$  (NSt), MSR applies the strategy **ViaDSt** (lines 8-16). When



**Fig. 11:** Two Examples of Strategy ViaDSt when NSt and DSt are in the same main area.

---

**Algorithm 5 MSR**


---

**Require:**  $E(S)$ ,  $E(D)$ ,  $E(St_1)$ ,  $\mathcal{ST}$ ,  $\mathcal{PO}$ ,  $\mathcal{ST}^{1W}$ ,  $\mathcal{ST}^{2D}$

**Ensure:**  $E(St)$

```

1: Initialize  $E(St) = \emptyset$ 
2: if  $\exists i \in [t]$ ,  $((PO_i[1] == E(St_1)) \wedge (PO_i[0] \text{ is unvisited}) \wedge (MA_{PO_i[0]} == MA_{PO_i[1]}))$  then //St1 is a SSt and it resides in same main area as PSt
3:   Insert  $\{E(St_1)\}$  into  $\mathcal{ST}^{1W}$ ;
4:    $E(St) \leftarrow \text{SK}$  in  $\mathcal{ST} \setminus \{E(St_1)\}$ , MSR; //1W: output the second-best stop in  $\mathcal{ST}$  omitting  $E(St_1)$ 
5: else if  $\exists i \in [t]$ ,  $((PO_i[1] == E(St_1)) \wedge (PO_i[0] \text{ is unvisited}) \wedge (MA_{PO_i[0]} >> MA_{PO_i[1]}))$  then //St1 is a SSt and it is in a lower main area than PSt
6:   Insert  $\{E(St_1)\}$  into  $\mathcal{ST}^{2D}$ ;
7:    $E(St) \leftarrow \text{SK}$  in  $\mathcal{ST} \setminus \{E(St_1)\}$ , MSR; //2D: output the second-best stop in  $\mathcal{ST}$  omitting  $E(St_1)$ 
8: else if  $\exists E(DSt)$  (NSt) between  $E(S)$  (CSt) and  $E(St_1)$  then //E(St1) is neither in  $\mathcal{ST}^{1W}$  nor in  $\mathcal{ST}^{2D}$ , ViaDSt:
9:   if  $MA_{E(DSt)}$  is visited: Return  $E(DSt)$ ;
10:  if  $MA_{E(DSt)} << MA_{E(St_1)}$ : Return  $E(DSt)$ ;
11:  else if  $MA_{E(DSt)} == MA_{E(St_1)}$ 
12:    Call partial C-AC to get DR;
13:    if  $DR == L$ 
14:      Invoke SK with  $C$ ;
15:    else: Repeat lines 14-15 with  $AC$ ;
16:  end if
17: Return  $E(St)$ ;

```

---

the deviation node  $DSt$  is in a visited main area, which has no unvisited stop, or it is in a main area that is lower than  $St_1$ , we MSR informs its caller to hop to  $DSt$ . If  $DSt$  and  $St_1$  are in the same main area (lines 11-16), MSR computes the skimming direction and then skims from  $CSt$ .

Now we explain the navigation rationale of NMid with Fig. 12. The left sub-figure shows the original VHN that navigates from  $S$  into  $A^{Mid}$ . However, the algorithm is flawed in its initial design because the navigation into  $A^{Mid}$  should be started from the last visited stop in  $A^{Low}$ . Such an observation dries us to come up with the right sub-figure, where we put forth the NMid to group via  $(St_1, D)$  and then invoke VNH.

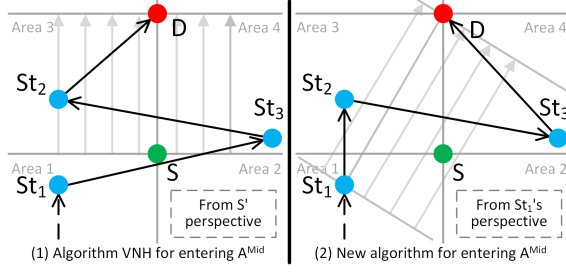


Fig. 12: An Example of Entering  $MA^M$ .

By doing so, we can improve the navigation accuracy.

We show the **algorithm NMid** in Algorithm 6. It first groups the remaining unvisited stops. If the low main area has unvisited stops, we invoke NMid to complete the navigation in  $MA^L$ . Next, NMid invokes  $\Sigma.VNH$  to compute the next hop  $E(St_{1st}^M)$ . Unlike the original VNH strategy in the  $MA^M$  of Hermes, which calculates the entry position from the starting point ( $S$ ) and often results in a longer (roundabout) path, Gaia's refined VNH computes this entry point in  $MA^M$  from the last visited stop in  $MA^L$ , ensuring a shorter (reasonable) path. Last, NMid recurses to Nav from  $E(St_{1st}^M)$  to  $E(D)$ .

#### Algorithm 6 NMid

**Require:**  $\mathcal{ERG}$ ,  $E(St_{last}^L)$ ,  $E(D)$ ,  $E(S)$ ,  $\mathcal{ST}^{Mid}$ ,  $\mathcal{EQR}$   
**Ensure:**  $ES_{1st}^M$

- 1: **If**  $\mathcal{ST}^{Mid} == \emptyset$ : Return  $E(St_{last}^L)$ ;
- 2: **If**  $|\mathcal{ST}^{Mid}| == 1$ : execute line 8-9, return  $E(St) \in \mathcal{ST}^{Mid}$ ;
- 3: Initialize  $ES_{1st}^M$ ,  $\mathcal{ST}^L$ ,  $\mathcal{ST}^M$ ,  $\mathcal{ST}^H$  as  $\emptyset$ ;
- 4:  $(\mathcal{ST}^L, \mathcal{ST}^M, \mathcal{ST}^H) \leftarrow \Sigma.SG(E(St_{last}^L), E(D), \mathcal{ST}^{Mid})$ ;
- 5: **if**  $\mathcal{ST}^L \neq \emptyset$
- 6:   Initialize  $E(St_{last}^L) \leftarrow \text{NLow}(\dots)$ , go to line 5;
- 7:  $E(St_{1st}^M) \leftarrow \Sigma.VNH(E(St_{last}^L), E(D), \mathcal{ST}^M)$ ;
- 8: **if**  $E(St_{1st}^M) == E(D)$ :  $E(St_{1st}^M) = E(St_{last}^L)$ ;  $\mathcal{ST}^M == \emptyset$
- 9:  $Esp \cup = \{ (E(St_{1st}^M), sp_{SSt_{1st}^M} \oplus F(sk_3, h(r_{St_{1st}^M})), r_{St_{1st}^M}) \}$ ;
- 10:  $Edt \cup = Esd_{SSt_{1st}^M}$ ;
- 11: **Nav**( $E(St_{1st}^M)$ ,  $E(D)$ ,  $\mathcal{ST}^{Mid} \setminus \{E(St_{1st}^M)\}$ );

Finally, we come to the **algorithm NHigh** in Algorithm 7. It adopts strategy C-AC to observe the location of  $St_{last}^M$  and node existence in  $A^H$ 's two semispaces (lines 3-10).

#### G. Decryption

After receiving  $\mathcal{EQR} = (Esp, Edt)$ , the user decrypts it as follows. For each triad  $\gamma = (\mathcal{I}(St), sp \oplus F(sk_3, h(r_{st})), r_{st})$  in  $Esp$ , user recovers  $sp$  by retrieving the  $r_{st_p}$  corresponding to  $P(sk_1, v_{st})$  and computing  $sp = \gamma[1] \oplus F(sk_3, h(\gamma[2]))$ ; concatenate all recovered  $sp$  to form the complete navigation path  $sp = S-St_1-\dots-D$ . User decrypts  $Edt$  by computing  $HD(sk, Edt)$  to get the path distance  $dt$ .

### VI. SECURITY ANALYSIS

Now we formally prove the security of Gaia.

#### Algorithm 7 NHigh

**Require:**  $\mathcal{ERG}$ ,  $E(St_{last}^M)$ ,  $E(S)$ ,  $E(D)$ ,  $\mathcal{ST}^H$ ,  $\mathcal{EQR}$   
**Ensure:**  $ES_{1st}^H$

- 1: **If**  $\mathcal{ST}^H == \emptyset$ : Return  $E(St_{last}^M)$ ;
- 2: **If**  $|\mathcal{ST}^H| == 1$ : execute lines 11-12, return  $E(St) \in \mathcal{ST}^H$ ;
- 3: Repeat lines 4-10 of **NLow** with  $S_l^H$ ,  $S_r^H$ ;
- 4: **if**  $\Sigma.SO(E(S), E(D), E(St_{last}^M)) == L$  **then**
- 5:   **if**  $S_l^H = 1 \wedge S_r^H = 0$ : Invoke SK with  $C$ ;
- 6:   **else if**  $S_l^H = 0 \wedge S_r^H = 1$ : Invoke SK with  $AC$ ;
- 7:   **else if**  $S_l^H = 1 \wedge S_r^H = 1$ : Invoke SK with  $C$ ;
- 8:   **else**  $ES_{1st}^H = E(D)$ ;
- 9: **else** Repeat lines 5-7 with  $AC$ ,  $C$ ,  $AC$ ;
- 10: **end if**
- 11:  $Esp \cup = \{ (E(St_{1st}^H), sp_{SSt_{last}^M St_{1st}^H} \oplus F(sk_3, h(r_{St_{1st}^H})), r_{St_{1st}^H}) \}$ ;
- 12:  $Edt \cup = Esd_{SSt_{last}^M St_{1st}^H}$ ;
- 13: **Nav**( $E(St_{1st}^H)$ ,  $E(D)$ ,  $\mathcal{ST}^H \setminus \{E(St_{1st}^H)\}$ );

**Theorem 1.** If  $\mathcal{P}$  and  $\mathcal{F}$  are pseudo-random,  $\Omega$  is CPA-secure, then  $\Pi$  is adaptively  $(\mathcal{L}_{Init}, \mathcal{L}_{Query}, \mathcal{L}_{Access})$ -semantically secure in the random oracle model where  $\mathcal{L}_{Init}(\mathcal{ERG}) = (n, Mdt, n_{dev})$ .

*Proof.* We construct a simulator  $\mathcal{S}$  working as below. Given  $\mathcal{L}_{Init}$ , it samples:

$$\tau_{ji} \leftarrow \{0, 1\}^{2|HE|+l} \text{ for all } i \in [n], j = 0,$$

$$\tau_{ij} \leftarrow \{0, 1\}^{\log n + 2|HE|+l} \text{ for all } 1 \leq i \leq n, 1 \leq j \leq n.$$

Next, it stores  $\zeta_i$  in a  $n+1$ -element array  $\mathcal{AR}_i$ . For all  $1 \leq i \leq n$ , it samples  $loc_i \leftarrow \{0, 1\}^{\log n}$  without repetition and sets  $\mathcal{RGD}[loc_i] \leftarrow \{0, 1\}^{\log n + l}$ .

Given  $\mathcal{L}_{Query}$  and  $\mathcal{L}_{Access}$ ,  $\mathcal{S}$  checks whether any of  $S$ ,  $D$ , and  $k$  stops  $\{St_1, \dots, St_k\}$  appeared in any previous navigation query. If  $S$  did,  $\mathcal{S}$  sets  $E(S) = (E(S)[0], E(S)[1])$  to previous values. If not, it sets  $E(S)[0] = loc_i$  for a unoccupied  $E(S)[1]$  and as below. It randomly selects a previously unused  $\alpha \in [n]$ , a key  $k_v \leftarrow \{0, 1\}^l$ , and sets  $E(S)[1] = \mathcal{EGD}[E(S)[0]] \oplus \langle \alpha || k_v \rangle$ . It records the association between  $k_S$  and  $E(S)[0]$  and  $\mathcal{ERD}[E(S)[0]]$ . It does the same for  $D$  and  $k$  stops analogously.

$\mathcal{S}$  simulates  $H$  as below. Given  $(k_v, r)$  as input,  $\mathcal{S}$  checks whether: (1)  $k_v$  has been queried previously; (2) if any entry in  $\mathcal{AR}$  has the form  $\langle str, r \rangle$  where  $str$  is a  $(\log n + 2|HE|)$ -bit string. If  $k_v$  is queried previously,  $\mathcal{S}$  creates a counter  $ct_{k_v} = 0$ . If an appropriate entry exists in  $\mathcal{AR}$ ,  $\mathcal{S}$  returns  $str \oplus \langle a_1, a_2, a_3 \rangle$  where  $a_1$  is a random path XORed with  $F(sk_4, r)$ , both  $a_2$  and  $a_3$  are  $HE(pk, 0)$ . If no appropriate entry exists,  $\mathcal{S}$  returns a random value. Then, Theorem 1 follows from the pseudo-randomness of  $\mathcal{P}$  and  $\mathcal{F}$  and the CPA-security of  $\Omega$ .

### VII. PERFORMANCE EVALUATION

In this section, we evaluate Gaia across four dimensions: path dilation rate, path similarity, computational cost, communication overhead. Furthermore, we compare its against SOTA Hermes.

### A. Experimental Settings

**Datasets and Parameters.** We utilize a real-world city map of Los Angeles extracted from OpenStreetMap, comprising 2,200 intersections, 986 roads, and 6,448 road segments. The key experimental parameters are summarized as follows: the number of road network nodes  $N$  ranges from 400 to 2,200; the number of stops  $k$  ranges from 0 to 10; the Blocked Edge Rate (BER) ranges from 0% to 20%; and the number of Partial Order (PO) constraints ranges from 0 to 5. To mitigate the impact of randomness in selecting blocked edges and partial orders, we conduct each experiment five times and report the average results.

**Baselines.** We adopt Hermes as the primary baseline, for its superiority in secure SCN. Since Hermes does not support blocked edges or partial order constraints, we compare it against Gaia in an ideal setting (BER=0%, PO=0). Additionally, we evaluate Gaia variants under varying BER = [5%, 15%] and PO = [1, 3, 5] to assess the algorithm's robustness in complex road conditions. A direct comparison with Hermes in complex settings is technically infeasible because Hermes inherently lacks the functionality to handle Edge Blockages or Partial Orders, thereby failing to generate valid paths.

**Metrics.** We evaluate: (1) **Path Dilation Rate (PDR)**: the ratio of the generated path length to the theoretical shortest distance, measuring distance overhead; (2) **Path Similarity (PS)**: a hybrid metric combining geometric grid alignment and length ratio to evaluate trajectory quality; (3) **Computational Cost**: time consumption for offline preprocessing, token generation, and online navigation; (4) **Communication Overhead**: data volume exchanged during graph upload, query submission, and result retrieval.

**Setup.** The NSP is instantiated on a computer equipped with an Intel Core i7-1165G7 (2.80GHz) processor and 32GB RAM. To support high-precision geographic coordinate calculations and improve engineering efficiency, we implemented the Gaia prototype using the CKKS homomorphic encryption scheme, replacing the BGN scheme used in Hermes. While CKKS ciphertexts ( $\approx 2KB$ ) are larger than BGN's, it uniquely supports floating-point operations and SIMD (Single Instruction Multiple Data) batching for processing multiple coordinates simultaneously. Crucially, BGN decryption requires solving the discrete logarithm ( $O(\sqrt{M})$ ), whereas CKKS decryption is a highly efficient polynomial operation ( $O(1)$ ), reducing latency from seconds to milliseconds. Key parameters are set as follows: the hash function uses SHA-256 (32 bytes), and the random oracle is instantiated with HMAC-SHA256. The CKKS ciphertext size is set to approximately 2KB to balance security and transmission efficiency, and the Garbled Circuit (GC) interaction overhead is approximately 4KB per round. For practical deployment, we give details in our Github webpage. [29].

### B. Path Dilation Rate

We define  $PDR = |dist|/|sd|$ . In the preprocessing, PDR is defined as the ratio of the path length generated by Gaia in a blocked graph to the theoretical shortest path length in

an unblocked graph. Since PO are dynamic query inputs, they do not impact the static graph preprocessing. Thus, Fig. 13 evaluates preprocessing PDR against BER,  $k$  and  $N$  only.

As shown in Fig. 13, Gaia not only possesses theoretical optimality in ideal environments but also demonstrates strong geometric robustness when network topology degrades heavily. Specifically, in a blockage-free scenario (BER=0%), PDR strictly converges to 1.0, confirming that the algorithm itself introduces no extra path overhead. Facing severe road network fragmentation (BER rising to 20%), Gaia effectively controls the PDR at a low level (generally  $< 1.5$ ), avoiding blind global detours. This efficiency stems from the enhanced blockage handling logic during preprocessing, which patches broken links by pre-computing local DSt, thereby exchanging minimal local cost for global connectivity. Furthermore, with increases in  $k$  and  $N$ , the PDR does not show divergent trends, and the curves for  $N = 1000$  and  $N = 2000$  exhibit high consistency. This superior scalability is attributed to the core MSR mechanism, which resolves complex spatiotemporal constraint conflicts via local fine-tuning.

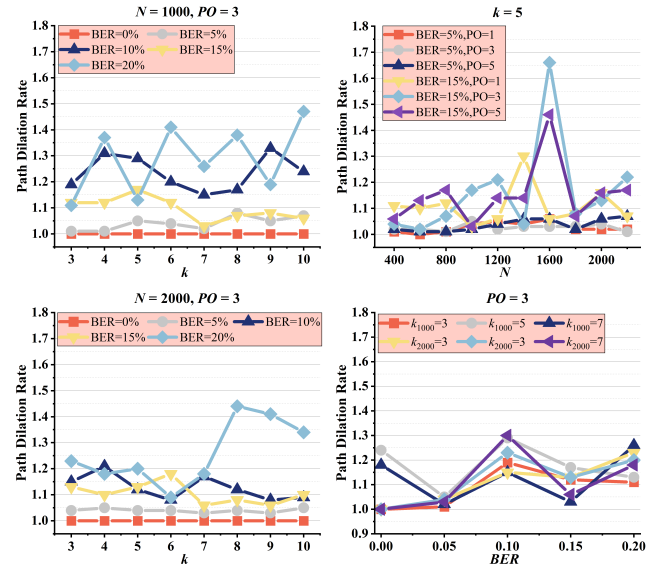


Fig. 13: Path Dilation Rate in preprocessing.

In navigation, we define PDR as the ratio of the path length generated by Gaia to the length of the shortest path obtained from a full permutation of stops. As shown in Fig. 14, although PDR trends upward with increasing  $k$  due to the combinatorial explosion of the search space, the overall ratio is largely controlled below 1.5. Most notably, as PO increases, PDR actually decreases, approaching 1.0 at PO=5. This phenomenon profoundly validates the effectiveness of the MSR mechanism: when constraints are few (PO=0), the theoretical optimal solution often utilizes geometric advantages to significantly shorten the path, leading to a relative deviation in Gaia; however, as constraints increase, the solution space is drastically compressed, allowing Gaia to precisely lock onto the unique or few feasible solutions satisfying the



temporal logic, thus achieving "geometric alignment" with the theoretical optimal solution.

Regarding the average overhead introduced by Deviation Stops against random and structured closures, our PDR effectively remains  $< 1.15$ . This confirms that the path length increase is strictly controlled within 15%, universally resolving connectivity with minimal overhead.

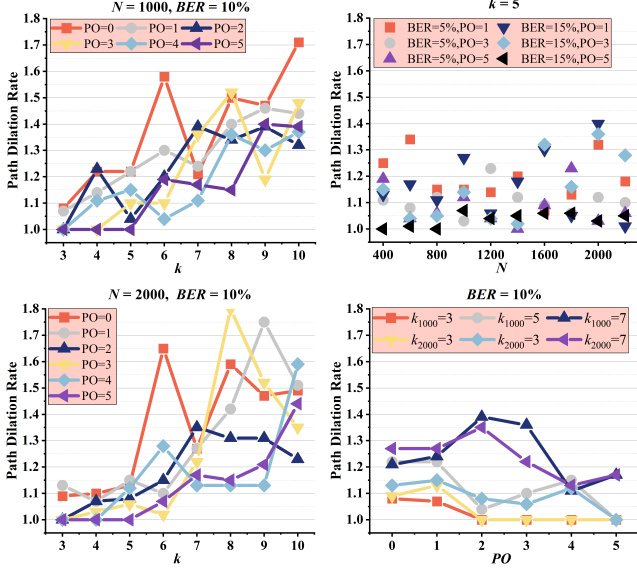


Fig. 14: Path Dilation Rate in the navigation.

### C. Path Similarity

The original node-hit ratio metric used in *Hermes* is no longer suitable for *Gaia*, because handling blocked edges and partial order constraints inevitably leads to the addition of nodes or sequence reordering. Therefore, we propose a new Hybrid Path Similarity (HPS) metric, defined as  $HPS = 0.7 \times \text{GridSim} + 0.3 \times \text{LenSim}$ . Specifically, HPS measures the similarity between the path generated by *Gaia* and the theoretical shortest path obtained through a full permutation of stops. **Grid Similarity (GridSim):** We partition the map into uniform  $50 \times 50$  meter grids and discretize the path into a grid frequency vector  $V = [v_1, \dots, v_n]$ . We calculate the spatial overlap between the actual path and the optimal path using the cosine similarity formula:  $(V_{\text{act}} \cdot V_{\text{opt}}) / (\|V_{\text{act}}\| \times \|V_{\text{opt}}\|)$ . **Length Similarity (LenSim):** This is the ratio of the optimal distance to the actual distance. We assign a higher weight (0.7) to spatial geometric features to prioritize evaluating the path's spatial orientation and trajectory fidelity, followed by length efficiency (0.3).

The HPS in the preprocessing is shown in Fig. 15. Under the ideal baseline of  $BER = 0\%$ , HPS stabilizes at 100%. As the blockage rate rises to 20%, although the physical path length inevitably increases, the overall HPS maintains above 80%. This implies that the computed trajectory strictly adheres to the optimal topology for necessary obstacle avoidance, without incurring directional deviations. Furthermore, HPS does not

significantly drop with increased  $PO$ , consistent with the reasons described in the PDR analysis.

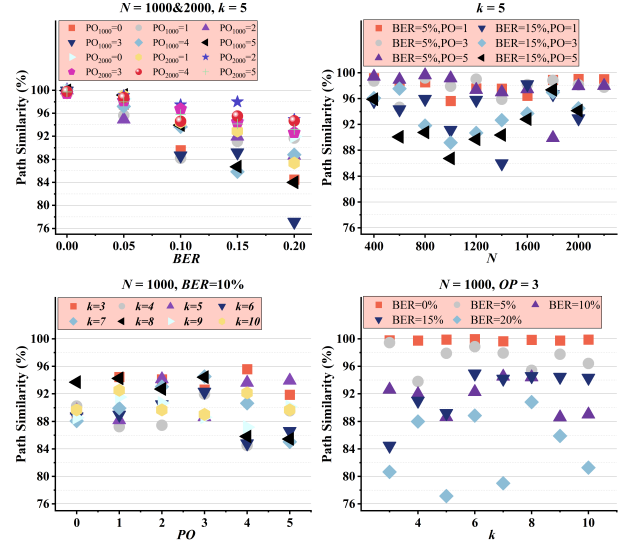


Fig. 15: Path Similarity in the preprocessing under varying  $BER$  and  $PO$ .

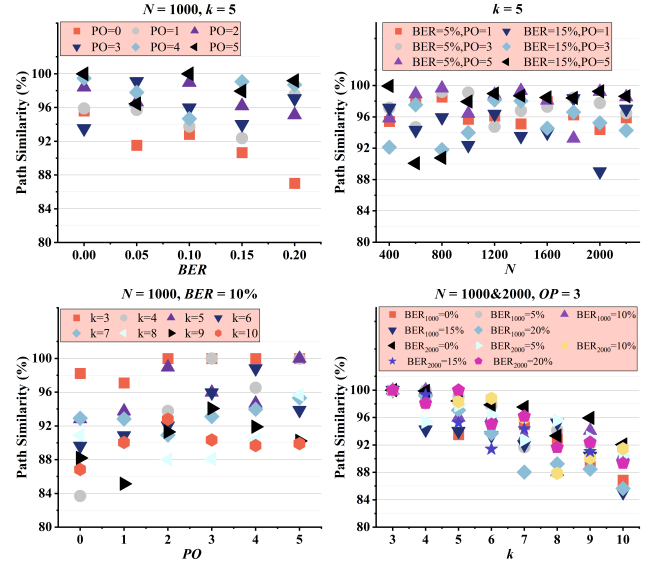


Fig. 16: Path Similarity in the navigation.

The HPS in navigation is shown in Fig. 16. Across various experimental conditions, HPS is generally guaranteed to be above 80% and mostly clusters above 90%. Although HPS decays linearly with the increase of stops  $k$ , the results remain controllable even in large-scale complex tasks with  $k = 10$  and  $N = 2000$ . This confirms that *Gaia* effectively suppresses error divergence when handling long-sequence navigation tasks.

The HPS metric relies on a weighting factor to balance Grid similarity and length similarity. To demonstrate the robustness of this metric and justify our default choice (0.7/0.3), we conducted a comprehensive sensitivity analysis. Fig. 17 illustrates the HPS performance across varying values from

0.1 to 0.9, encompassing both the online navigation phase and the offline preprocessing phase. The results, averaged over networks ranging from 400 to 2200 nodes under constraints ( $k = 5$ ,  $PO = 3$ ,  $BER = 0.05$ ), indicate that HPS remains remarkably stable across a wide spectrum of weight assignments (all  $> 80\%$ , most  $> 90\%$ ). Specifically, although the adjustment factor does affect HPS, most results remain within a very high range, confirming that our algorithm is not overly sensitive to precise parameter tuning and reliably produces paths analogous to ideal unconstrained trajectories.

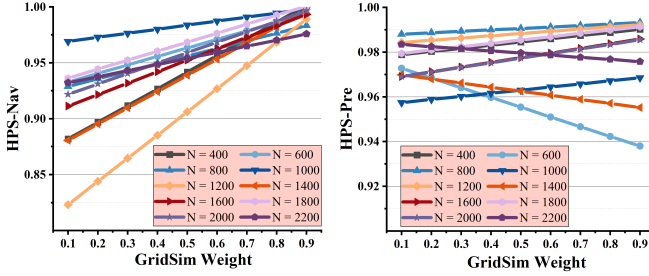


Fig. 17: Sensitivity analysis of HPS metrics during navigation and preprocessing ( $k = 5$ ,  $PO = 3$ ,  $BER = 0.05$ ).

#### D. Computational Efficiency

**Preprocessing Efficiency.** Fig. 18 illustrates the computational cost of blocked edge preprocessing, derived from measuring the execution time of our enhanced Floyd-Warshall framework. This framework handles blockage by sequentially employing a three-layer strategy: initially performing a Direct DSt Search to screen candidate nodes, followed by a Neighbor Path Query utilizing precomputed results, and finally triggering History Backtracking for re-planning when necessary (please see full version [29]). The processing time increases linearly with  $N$ . Even under extreme conditions of  $N = 2200$  and  $BER = 15\%$ , the processing time remains approximately 550 ms, demonstrating ms-level response speed. The figure also displays the ratio of Blocked Edge Time to total initialization time. Notably, as  $N$  expands, this ratio drops sharply: from 8%-12% in small-scale networks to rapidly converging and stabilizing below 2%. This convergence highlights the algorithm's extremely low marginal cost and significant superiority in large-scale scenarios. Fig. 18 illustrates the computational cost of blocked edge preprocessing, derived from measuring the execution time of our enhanced Floyd-Warshall framework. This framework handles blockage by sequentially employing a three-layer strategy: initially performing a Direct DSt Search to screen candidate nodes, followed by a Neighbor Path Query utilizing precomputed results, and finally triggering History Backtracking for re-planning when necessary (please see full version [29]). The processing time increases linearly with  $N$ . Even under extreme conditions of  $N = 2200$  and  $BER = 15\%$ , the processing time remains approximately 550 ms, demonstrating ms-level response speed. The figure also displays the ratio of Blocked Edge Time to total initialization time. Notably,

as  $N$  expands, this ratio drops sharply: from 8%-12% in small-scale networks to rapidly converging and stabilizing below 2%. This convergence highlights the algorithm's extremely low marginal cost and significant superiority in large-scale scenarios. Furthermore, to handle ultra-large ( $> 100,000$ ) deployments and mitigate the  $O(N^3)$  preprocessing limits, by employing Graph Partitioning algorithms (e.g., METIS), massive maps can be decomposed into subgraphs for parallel preprocessing, significantly reducing complexity.

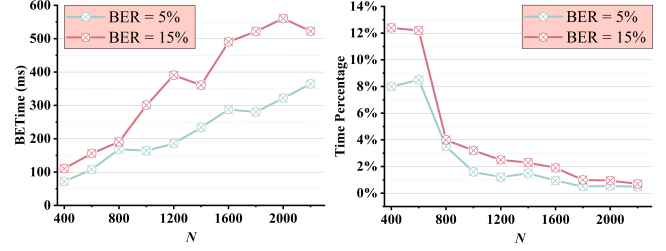


Fig. 18: Preprocessing efficiency and BeTime ratio.

**Online Navigation Efficiency.** When  $N = 800$ , the navigation time performance under different blockage rates and partial order constraints is shown in Fig. 19 and Fig. 20. Fig. 19 demonstrates a stable linear growth as  $k$  increases, confirming that Gaia successfully avoids the exponential cost of full stops permutation. This efficiency stems from our hierarchical planning strategy, which narrows the search space to local areas and determines the visitation order through geometric heuristics rather than exhaustive enumeration. Although high blockage rates inevitably increase time, performance fluctuations are minimal under low-to-medium blockage rates (0%–15%). In Fig. 20, under identical  $k$  and  $BER$  conditions, increasing the number of constraints does not significantly increase computation time. This is mainly attributed to two points: first, the MSR strategy finds safe alternative nodes in constant time, avoiding complex backtracking; second, the DaC strategy limits constraint impact to a local scope, avoiding global re-planning.

#### E. Communication and Computation Overhead

We deconstruct the total communication overhead of Gaia ( $C_{\text{total}}$ ) into offline preprocessing ( $C_{\text{off}}$ ) and online query ( $C_{\text{on}}$ ) components. Let  $|C_{\text{tx}}|$  be the ciphertext size and  $S_{\text{gc}}$  be the interaction size of a single GC round.

**Offline Phase.** Uploading encrypted data is a one-time cost:  $C_{\text{graph}} = |PK| + 2 \cdot n \cdot |C_{\text{tx}}|$ . **Online Phase.** The overhead includes token upload, result download, and interaction:  $C_{\text{token}} = (k + 2)(2|C_{\text{tx}}| + \delta)$ ,  $C_{\text{res}} = (k + 1)(|C_{\text{tx}}| + \gamma)$ , and  $C_{\text{inter}} = (PO + \alpha)S_{\text{gc}}$ . Total cost scales linearly:  $C_{\text{on}} = O(k|C_{\text{tx}}|) + O(PO \cdot S_{\text{gc}})$ , ensuring scalability. Where  $\delta$  and  $\gamma$  are constant sizes for auxiliary identification information, and  $\alpha$  is the base number of interaction rounds. It is evident that the online communication overhead of Gaia grows linearly with query complexity  $k$  and  $PO$ , demonstrating good scalability.

Table III shows the actual communication overhead for different numbers of stops  $k$  at a node scale of  $N = 1000$ .

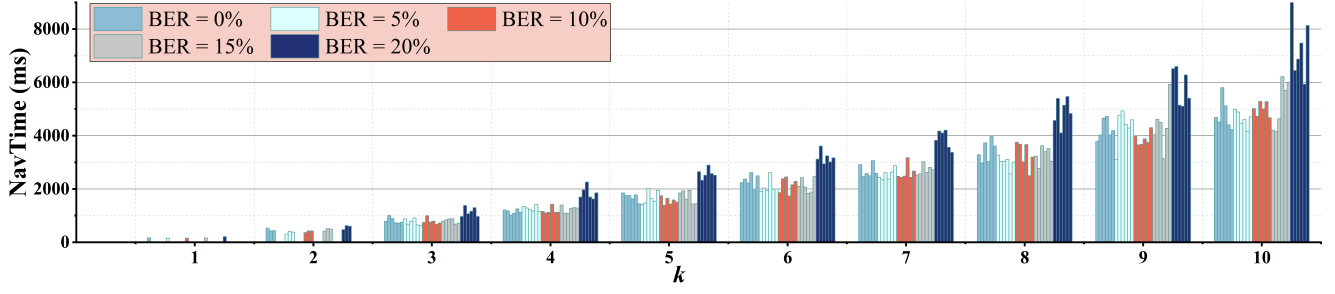


Fig. 19: Navigation time under varying  $BER$  and  $k$  ( $N = 800$ ).

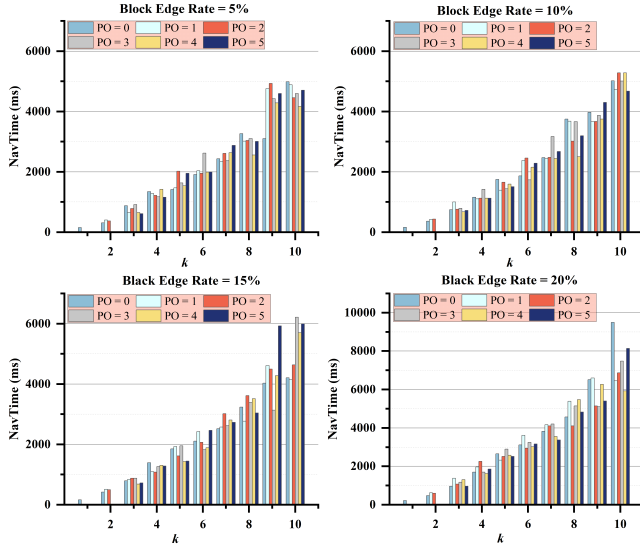


Fig. 20: Impact of  $PO$  on navigation time ( $N = 800$ ).

TABLE III: Online Communication Overhead of Gaia (Unit: KB)

| $k$ | $C_{token}$ | $C_{res}$ | $C_{inter}$ | $C_{online}$ |
|-----|-------------|-----------|-------------|--------------|
| 1   | 12.18       | 6.12      | 8.00        | 26.3         |
| 2   | 16.25       | 8.25      | 8.00        | 32.5         |
| 3   | 20.31       | 10.37     | 8.00        | 38.7         |
| 4   | 24.37       | 12.50     | 8.00        | 44.9         |
| 5   | 28.43       | 14.62     | 8.00        | 51.0         |
| 6   | 32.50       | 16.75     | 8.00        | 57.2         |
| 7   | 36.56       | 18.87     | 8.00        | 63.4         |
| 8   | 40.62       | 21.00     | 8.00        | 69.6         |
| 9   | 44.68       | 23.12     | 8.00        | 75.8         |
| 10  | 48.75       | 25.25     | 8.00        | 82.0         |

As shown in Fig. 21, both communication and computation overheads of Gaia increase linearly with  $k$ , exhibiting excellent scalability. Even under a complex query with  $k = 10$ , the total communication volume is controlled at 82 KB. The computational latency on the client side primarily stems from token generation. The data indicates that generating a query token for 10 stops takes less than 20 ms.

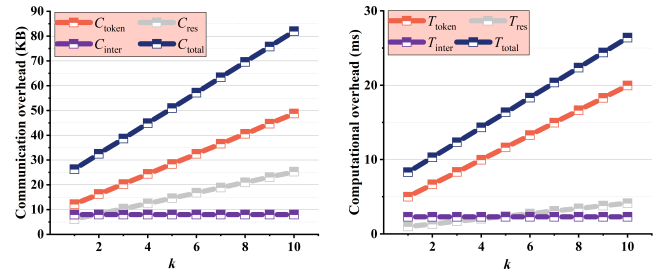


Fig. 21: Communication and computation overhead with varying  $k$ .

**Memory and Time Breakdowns:** The memory footprint of the encrypted index grows linearly. For  $N = 2200$  under a 15% blockage rate, the encrypted road graph ( $ERG$ ) strictly consumes only 485 KB of cloud storage. Furthermore, generating a query token for  $k = 10$  stops requires  $\leq 152$  ms (accounting for  $< 10\%$  of the total 1.6 s). The server-side query time scales strictly at  $O(k)$ , maintaining a stable execution latency regardless of the graph size  $N$ .

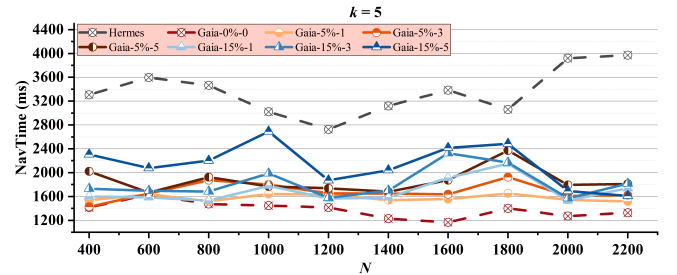


Fig. 22: Comparison of navigation efficiency between Gaia and Hermes across different  $N$ .

## F. Comparison

Finally, Fig. 22 compares the efficiency of Gaia and Hermes ( $k = 5$ ) across varying network sizes. First, Gaia consistently outperforms Hermes. Across all test scales ( $N = 400 \sim 2200$ ), Hermes' latency remains significantly higher (peaking near 4000 ms), while all Gaia variants lie well below it. Specifically, in the ideal case (Gaia-0%-0), Gaia maintains

a time of approximately 1200 ~ 1400 ms, achieving a substantial reduction compared to Hermes. Second, although latency increases with BER and PO, Gaia exhibits strong robustness. Even under the most stringent conditions (i.e., Gaia-15%-5), its time remains lower than the Hermes baseline. This indicates that Gaia successfully balances functional enhancement and speed without sacrificing system responsiveness.

### VIII. CONCLUSIONS

In this work, we proposed Gaia, a secure semi-constrained navigation framework that addresses the limitations of existing schemes regarding partial visit orders and geographic obstacles. By leveraging a refined Divide-and-Conquer strategy and Homomorphic Encryption with GC and OT, Gaia efficiently handles Multiple Spatial Restrictions while protecting location privacy. Formal analysis proves its security under the semi-honest model. Extensive experiments on real-world datasets demonstrate that Gaia significantly outperforms the SOTA baseline, Hermes. Furthermore, the evaluation results confirm that Gaia maintains precise **geometric alignment** with optimal trajectories, exhibiting robustness and scalability even in complex and obstacle-dense environments.

### IX. FUTURE WORK

Moving forward, we identify three critical directions to enhance the practical deployment of Gaia. First, to mitigate the risk of a compromised semi-honest proxy, a promising future direction is to explore a Secure Multiparty Computation model (e.g., 2-Party Computation) [31], which splits the secret key between the Proxy and the server to require collaborative decryption. Second, exploring the execution of critical computations within a Trusted Execution Environment (TEE, e.g., Intel SGX) could provide an additional hardware-level security layer. Finally, adapting to real-time dynamic obstacles (e.g., traffic jams) necessitates dynamic updates to graph encryption. Our future work will explore dynamically updating the encryption index directly on ciphertexts, pre-dividing the graph into modular subgraphs, and updating only the weights/blocking states of the affected edges, rather than performing costly full graph re-encryption.

### REFERENCES

- [1] S. Lai, X. Yuan, S.-F. Sun, J. K. Liu, Y. Liu, and D. Liu, "GraphSE<sup>2</sup>: An encrypted graph database for privacy-preserving social search," *Proc. 14th ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, July 2019: 41-54, Auckland, New Zealand.
- [2] Y. Peng, Z. Ma, W. Zhang, X. Lin, Y. Zhang, and X. Chen, "Efficiently answering quality constrained shortest distance queries in large graphs," *Proc. 39th IEEE International Conference on Data Engineering (ICDE)*, 2023: 856-868, Anaheim, USA.
- [3] T. Yu, J. Liu, Y. Yang, Y. Li, H. Fei, and P. Li, "EGM: Enhanced Graph-based Model for Large-scale Video Advertisement Search," *Proc. 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022: 4443 - 4451, Washington, USA.
- [4] P. Zhao, J. Yu, H. Zhang, Z. Qin and C. Wang, "How to securely outsource finding the min-cut of undirected edge-weighted graphs," *IEEE Transactions on Information Forensics and Security (TIFS)*, 2020, 15: 315-328.
- [5] K.-T. Foerster, J. Hirvonen, Y.-A. Pignolet, S. Schmid and G. Tredan, "On the Price of Locality in Static Fast Rerouting," *Proc. 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2022: 215 - 226, MD, USA.
- [6] Z. Liu, S. Shekhar and C. Peng, "Breaking Geographic Routing Among Connected Vehicles," *Proc. 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2023: 42 - 54, Porto, Portugal.
- [7] X. Meng, S. Kamara, K. Nissim, and G. Kollios, "GRECS: Graph encryption for approximate shortest distance queries," *Proc. 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, October, 2015: 504-517, Denver, USA.
- [8] D. J. Wu, J. Zimmerman, J. Planul, and J. C. Mitchell, "Privacy-preserving shortest path computation," *Proc. 23rd Annual Network and Distributed System Security Symposium (NDSS)*, February, 2016, San Diego, USA.
- [9] J. G. Chamani, I. Demertzis D. Papadopoulos, C. Papamanthou, and R. Jalili, "GraphOS: Towards Oblivious Graph Processing," *Proc. 49th International Conference on Very Large Data Bases (VLDB)*, 2023, 16(13): 4324 - 4338, 2023.
- [10] New York Times, "Twelve Million Phones, One Dataset, Zero Privacy," <https://www.nytimes.com/interactive/2019/12/19/opinion/location-tracking-cell-phone.html>, 2019.
- [11] Q. Wang, K. Ren, M. Du, Q. Li, and A. Mohaisen, "Secgdb: Graph encryption for exact shortest distance queries with efficient updates," *Proc. 21st International Conference on Financial Cryptography and Data Security (FC)*, April 2017: 3-7, Sliema, Malta.
- [12] E. Ghosh, S. Kamara, and R. Tamassia, "Efficient graph encryption scheme for shortest path queries," *Proc. 16th ACM Asia Conference on Computer and Communications Security (ASIACCS)*, June 2021: 516-525, Hong Kong, China.
- [13] M. Du, S. Wu, Q. Wang, D. Chen, P. Jiang, and A. Mohaisen, "Graphshield: Dynamic large graphs for secure queries with forward privacy," *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 2022, 34 (7): 3295-3308.
- [14] F. Wang, Z. Chen, L. Pan, L. Y. Zhang, and J. Zhou, "CryptGraph: An efficient privacy-enhancing solution for accurate shortest path retrieval in cloud environments," *Proc. 19th ACM Asia Conference on Computer and Communications Security (ASIACCS)*, 2024, 1660-1674, Singapore, Singapore.
- [15] F. Falzon, E. Ghosh, K. G. Paterson, and R. Tamassia, "PathGES: An efficient and secure graph encryption scheme for shortest path queries," *Proc. 31st ACM SIGSAC Conference on Computer and Communications Security (CCS)*, October 2024: 4047-4061, Salt Lake City, USA.
- [16] C. Liu, L. Zhu, X. He and J. Chen, "Enabling privacy-preserving shortest distance queries on encrypted graph data," *IEEE Transactions on Dependable and Secure Computing (TDSC)*, 2021, 18 (1): 192-204.
- [17] M. Li, Y. Chen, J. Gao, J. Wu, Z. Zhang, J. He, L. Zhu, M. Conti, and X. Lin, "Accurate, secure, and efficient semi-constrained navigation over encrypted city maps," *IEEE Transactions on Dependable and Secure Computing (TDSC)*, 2024, Online.
- [18] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," *Proc. 23rd International Conference on the Theory and Applications of Cryptology and Information Security (ASIACRYPT)*, December, 2017: 409-437, Hong Kong, China.
- [19] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," *Proc. 23rd International Conference on the Theory and Applications of Cryptology and Information Security (ASIACRYPT)*, December 2017: 409-437, Hong Kong, China.
- [20] M. Chase and S. Kamara, "Structured encryption and controlled disclosure," *Proc. 16th International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, December 2010: 577-594, Singapore.
- [21] E. Cohen, E. Halperin, H. Kaplan, and U. Zwick, "Reachability and distance queries via 2-hop labels," *Proc. 13th ACM-SIAM Symposium on Discrete Algorithms (SODA)*, January 2002: 937-946, San Francisco, USA.
- [22] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, "Searchable symmetric encryption: Improved definitions and efficient constructions," *Proc. 13th ACM conference on Computer and Communications Security (CCS)*, October 2006: 79-88, Alexandria, USA.
- [23] M. Chase and S. Kamara, "Structured encryption and controlled disclosure," *Proc. 16th International Conference on the Theory and Ap-*



*plication of Cryptology and Information Security (ASIACRYPT)*, 2010: 577-594, Singapore.

- [24] A. Yao, "Protocols for secure computations (extended abstract)," *Proc. 23rd Annual Symposium on Foundations of Computer Science (FOCS)*, November 1982: 160-164, Chicago, USA.
- [25] A. Yao, "How to generate and exchange secrets (extended abstract)," *Proc. 27th Annual Symposium on Foundations of Computer Science (FOCS)*, October 1986: 162-167, Toronto, Canada.
- [26] M. Naor and B. Pinkas, "Efficient oblivious transfer protocols," *Proc. 20th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, January, 2001: 448-457, Washington, USA.
- [27] D. Boneh, E. Goh, and K. Nissim, "Evaluating 2-dnf formulas on ciphertexts," *Proc. 2nd Theory of Cryptography, Second Theory of Cryptography Conference (TCC)*, February 10-12, 2005: 325-341, Cambridge, USA.
- [28] R. W. Floyd, "Algorithm 97: Shortest path," *Communications of the ACM*, 1962, 5 (6): 344-348.
- [29] (2025) Full version, <https://github.com/NavTest1/Gaia1>.
- [30] J. Katz and Y. Lindell, "Introduction To Modern Cryptography (Third edition)," *Chapman & Hall/CRC Cryptography and Network Security (51 books)*, 2021.
- [31] Z. Huang, W. Lu, C. Hong, and J. Ding, "Cheetah: Lean and Fast Secure Two-Party Deep Neural Network Inference," *Proc. 31st USENIX security symposium (USENIX Security 22)*, August 2022: 809-826, Boston, USA.